

UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

DIPARTIMENTO DI INGEGNERIA "ENZO FERRARI"
CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

Evaluation of Continual Learning approaches to mitigate mode collapse in Generative Adversarial Networks

Candidato:

Lorenzo Schioli

Relatore:

Prof. Simone Calderara

Correlatori:

Dott. Lorenzo Bonicelli

Dott. Matteo Boschini

Dott. Angelo Porrello

Anno accademico 2020-2021

Sommario

Quello dell'Intelligenza Artificiale è uno dei temi più in voga degli ultimi anni. Al giorno d'oggi, l'Intelligenza Artificiale è già utilizzata in diversi campi, è piuttosto performante e ha grandi potenzialità, ma ha ancora diversi margini di miglioramento. Il presupposto fondamentale per addestrare gli algoritmi di Machine Learning è l'I.I.D.: i dati dovrebbero essere indipendenti e distribuiti in modo identico. Tuttavia, questa ipotesi è limitante poiché, nel mondo reale, i dati di addestramento non sono sempre campionati casualmente dalla distribuzione, ma il campionamento potrebbe essere focalizzato su una porzione della distribuzione e questa porzione potrebbe cambiare nel tempo. Gli algoritmi di apprendimento automatico standard non sono progettati per funzionare senza il vincolo dell'I.I.D.; infatti, non funzionano correttamente senza di esso, causando il problema del Catastrophic Forgetting: dopo essersi allenati per un po' di tempo iniziano a dimenticare ciò che hanno imparato in precedenza. Il Continual Learning è un campo dell'apprendimento automatico che considera questo problema e cerca di risolverlo, fornendo agli algoritmi di apprendimento automatico la capacità di apprendere continuamente nuovi dati senza dimenticare quelli vecchi.

In questo lavoro affronteremo il comune problema del Mode Collapse nelle GAN, che in questo caso sembra essere correlato a un problema di Catastrophic Forgetting del discriminatore. Poiché, in presenza di Mode Collapse, il generatore cambia modalità più volte, l'input del discriminatore diventa non I.I.D., e quindi il suo addestramento può essere visto come un problema di Continual Learning, che porta a un possibile Catastrophic Forgetting. Di conseguenza, cercheremo di far funzionare correttamente la nostra GAN applicandogli metodi di Continual Learning.

Abstract

Artificial Intelligence is one of the hottest topics of recent years. Nowadays, Artificial Intelligence is already used in several fields, is quite powerful, has great potentials, but it has still several margins of improvement. The fundamental assumption to train machine learning algorithms is the I.I.D. assumption: the data should be Independent and Identically Distributed. However, this assumption is limiting since, in the real world, training data is not always sampled randomly from distribution but the sampling could be focused on a portion of the distribution and this portion could change over time. Standard Machine learning algorithms are not designed to work without the I.I.D. constraint; indeed, they don't function properly without it, causing the problem of Catastrophic Forgetting: after training for some time they start to forget what they have previously learned. Continual Learning is a Machine Learning field that considers this problem and tries to solve it, providing the Machine Learning algorithms the ability to continuously learn new data without forgetting the old one.

In this work, we will deal with the common problem of Mode Collapse in GANs, which, in this case, seems to be correlated to a Catastrophic Forgetting problem of the discriminator. Since, in the presence of Mode Collapse, the generator changes its mode several times, the discriminator's input becomes non-I.I.D., and thus its training can be seen as a Continual Learning problem, leading to a possible Catastrophic Forgetting. Accordingly, we will try to make our GAN work correctly by applying Continual Learning methods to it.

Contents

Sommario	iii
Abstract	v
1 Introduction	1
2 Machine Learning and Deep Learning	3
2.1 Machine Learning	3
2.2 Deep Learning	15
3 Related Works	33
3.1 Avoiding mode collapse	33
3.2 Continual learning Methods	39
4 Case study	51
4.1 Mode Collapse and Catastrophic Forgetting in an eight Gaussians distribution	51
4.2 Implementation	54
5 Empirical Study	59
5.1 Evaluation methods	60
5.2 Comparison between approaches	62
5.3 Comparison between combinations	63
5.4 Varying Experience Replay's buffer size	64
6 Discussion	71
6.1 Considerations on the different approaches	71
6.2 Considerations on the combinations	72
6.3 Considerations on the ER buffer size	74
7 Conclusion	77
Bibliography	79

List of Figures

2.1	Bias and Variance	9
2.2	Fitting	10
2.3	Overfitting and Underfitting	11
2.4	Perceptron	16
2.5	Activation Functions	18
2.6	Activation Functions	19
2.7	Backpropagation	22
2.8	GAN	27
2.9	GAN loss	28
2.10	GAN loss	29
3.1	Fitting	34
3.2	WGAN gradient	37
3.3	WGAN schema	38
3.4	Catastrophic forgetting	41
3.5	EWC	45
4.1	Eight Gaussians	52
4.2	GAN training on eight Gaussians	53
5.1	GANs video	59
5.2	Evaluation	62
5.3	Evaluation GAN	65
5.4	Evaluation GAN-L2	66
5.5	Evaluation WGAN	66
5.6	Evaluation GAN-EWC	67
5.7	Evaluation GAN-ER	67
5.8	Evaluation GAN-L2-EWC	68
5.9	Evaluation WGAN-EWC	68
5.10	Evaluation GAN-L2-EWC	69
5.11	Evaluation WGAN-ER	69
5.12	Evaluation ER buffer	70

List of Tables

5.1	Sym-KL of the models	63
5.2	Sym-KL of the models' combinations	64
5.3	Sym-KL of ER with different buffer size	65

List of Abbreviations

AI	Artificial Intelligence
ANN	Artificial Neural Network
CL	Continual Learning
CNN	Convolutional Neural Network
ML	Machine Learning
MLP	MultiLayer Perceptron
MSE	Mean Squared Error
DL	Deep Learning
ER	Experience Replay
EWC	Elastic Weight Consolidation
IID	Independent and Identically Distributed
KL	Kullback-Leibler
kNN	k-Nearest Neighbors
LwF	Learning without Forgetting
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descend
SI	Synaptic Intelligence
SSL	Semi-Supervised Learning
GAN	Generative Adversarial Networks
WGAN	Wasserstein GAN
L2	L2 regularization
GAN-L2	GAN with L2
GAN-EWC	GAN with EWC
GAN-ER	GAN with ER
GAN-L2-EWC	GAN with L2 and EWC
GAN-L2-ER	GAN with L2 and ER
WGAN-EWC	WGAN with EWC
WGAN-ER	WGAN with ER
Sym-KL	Symmetric Kullback - Leibler divergence

1 Introduction

Artificial intelligence models have improved a lot in the past years, becoming quite capable nowadays. Artificial intelligence is used in several areas. In the security field, there are fingerprint scanners, face recognition, and security cameras that are able to detect unusual movements. In agriculture and farming AI is used in order to optimize the growth of plants or to reduce waste of resources, like water. In the car industry, AI is used for implementing self-driving cars. In general, it is used to simplify our lives and improve several technologies.

However AI is not perfect, and it will never be, so there is always a margin of improvement. One of the most important limits and problems of current models is the so-called catastrophic forgetting. It means that, as the model keeps training, it will reach a point in which it starts to forget what it has previously learned. Because of this, it's like that the current AIs have an absence of long-term memory, which is an important lack if we compare AI with human intelligence. This happens when there is a scenario where the model has to learn from a sequence of tasks, each one with a different distribution. This leads to a training dataset that isn't Independent and Identically Distributed (I.I.D.), which is a scenario not suited to the traditional Machine Learning algorithms. Continual Learning is the branch of AI that tries to solve the problem of catastrophic forgetting and give the AI the capability of continuously learning over time.

In this work, we will deal with a Generative Adversarial Networks (GAN) and the Mode Collapse, which is a well know problem of GANs and it seems to be correlated with the catastrophic forgetting problem and the Continual Learning scenario: each mode of Mode Collapse corresponds to a task of Continual Learning, where the tasks must be learned in sequence. This scenario leads to a training dataset that isn't I.I.D., bringing the model to catastrophic forgetting. This means that if we apply Continual Learning approaches to the GAN model we may limit the Mode Collapse problem and therefore improve the model's performance. In particular, the GAN is trained on a distribution that is a mixture of eight Gaussians, where, each time the generator overfits in

one of them, it corresponds to a mode of Mode Collapse.

In this work, we will introduce the basics of Machine Learning and Deep Learning in order to understand Generative Adversarial Networks and the environment where we will work. Then, we will explain the problems of Mode Collapse and Catastrophic Forgetting and we will see some approaches designed to solve them, like L2 regularization, Wasserstein GAN, Elastic Weight Consolidation, and Experience Replay. Afterward, we will focus on our specific GAN trained on the eight Gaussians distribution and we will see the implementation details. Next, we will have a sight at the results of the approaches used to mitigate the Mode Collapse and Catastrophic Forgetting problems. Finally, we will draw conclusions observing the results.

2 Machine Learning and Deep Learning

Machine Learning and Deep Learning are both study fields that belongs to the *Artificial Intelligence (AI)* topic. AI is often defined as the study of Intelligent Agents, like the following definition.

Artificial intelligence (AI) refers to systems that display intelligent behaviour by analysing their environment and taking actions, with some degree of autonomy, to achieve specific goals.

— AIEurope

In the following chapters we will explain ML and DL more in detail.

2.1 Machine Learning

Machine Learning (ML) is an AI field that studies algorithms where some parts of them are not determined by the programmer but by experience.

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

— Mitchell [32]

The performance measure is a measure of how well the program is behaving. The goal is to increase this measure, which is done by modifying the program through training (the learning process). The experience corresponds to the training data given in input to the program during the learning process.

There are several kinds of tasks, such as: *supervised*, *unsupervised* and *semi-supervised* learning [4].

Supervised Machine Learning

The name supervised learning comes from the fact that during training there is a phase of examination where the data labels are looked over to evaluate if the prediction is correct or not. This examination phase is like an oracle that supervises the algorithm through the learning process by checking if the algorithm is predicting well or not, driving it through a better prediction. The prediction is corrected by the supervisor. The learning process terminates when the algorithm reach an acceptable level of performance.

The main characteristic is that all the data is labeled and the machine learning algorithms learns by attempting a prediction and then checking the label to see if the prediction was correct or incorrect; based on this result, the algorithm changes "some of its parameters to improve the prediction.

Supervised learning is the most common task used in the majority of practical machine learning. From a mathematical point of view, if we consider an input variable X and a desired output variable Y , the goal of the algorithm is to learn the mapping function from the input to the output:

$$Y = f(X; \theta). \quad (2.1)$$

The goal is to learn the mapping function in such a way that, when training is finished, it is able to predict the correct output given new input data without labels.

Supervised learning problems can be further grouped into regression and classification problems:

- **Classification:** The task consists in providing in input to the model an object and it has to choose which is the class the object belongs. For example, we might provide some images of pets and ask the model if the picture is a cat or a dog. Two popular examples of classification are random forest and support vector machine.
- **Regression:** The task consists in predicting a continuous value given the an input. For example, if we give in input to the program some car features, it might have to predict the price. A popular examples of classification is linear regression.

In both cases, the goal is to estimate a relationship between the dependent variables (predicted value) and the independent variables (input).

The supervised machine learning are trained on a dataset that is called training set and then its performance are evaluated with the test set. A test set is basically a set of examples needed to only to evaluate the generalization capabilities (that is the performance) of a fully specified classifier. After the training set, the model learned to make class predictions on the training data and those predictions are then compared to the true classification of the test data to check the accuracy of the model. The test set is independent with respect to the training set, but they follow the follows the same probability distribution.

Unsupervised Machine Learning

Unsupervised Machine Learning It is called *unsupervised* since there is no checking of the labels and therefore there is no supervisor. Since there is no label it is not possible to directly estimate the real accuracy. Unsupervised Machine Learning is the opposite of the supervised one: training data don't have labels. Consequently, the goal changes with respect to the supervised setting and the objective is to find similarities between elements or structures (patterns), thus doing clustering. The goal can also be seen as the task of learning more about the data through the modeling of the underlying structure or distribution of the data.

Unsupervised learning problems can be further grouped into *clustering* and *association* problems.

- **Clustering:** The task consists in giving in input several objects and the program has to group them together (forming clusters) given some similarity between them. For example, if we give in input to the program several images of animals, it has to creates groups given some similarity like the color of the skin. A popular example is k-means.
- **Association:** An association rule learning problem is the attempt of discovering rules that describe portions of the data, for example the correlation between bought products (if a person buy X also tend to buy Y). A popular example is the a-priori algorithm.

Semi-Supervised Machine Learning

Semi-Supervised Machine Learning is the combination of the previous two: some data are labeled and other not (most of them are not). In this case, a

composition of supervised and unsupervised techniques can be used. From a mathematical point of view, there is the input data X and only some of it has its corresponding desired output Y (that is the label). Many real world machine learning problems fall into this area. Semi-Supervised Machine Learning is useful considering that it is quite expensive or time-consuming to label all the data (it is done manually) and there could be also the need of domain experts. On the other hand, unlabeled data is quite cheap and easier to collect and store.

Unsupervised learning techniques are used to find the structure of the input. An example is the *Self-supervision*: supervised learning is used to make predictions for the unlabeled part of the data; after the prediction, the data is used as training and fed back into the supervised learning algorithm to improve the prediction for new unseen data.

Reinforcement Learning

In **reinforcement learning**, the system learns by looking at a value that is called reward. If this value is high it means that the algorithm is doing well; while, on the other hand, if it is low (also called “punishment”) it means that the algorithm is not behaving well. The model’s goal is to behave in such a way that the reward is maximized [48]. It is called *reinforcement* since the good behaviours are reinforced by a positive reward. For example, if we apply a reinforcement learning algorithm to a video game, the reward would correspond to the game score. Looking at the score, the model can then identify which moves are good and which not, learning a strategy. The model will accumulate the highest reward possible.

Bias–variance tradeoff

Firstly we have to explain bias and variance since they are needed to understand two of the most common problems of machine learning algorithms: underfitting and overfitting (2.1) [22].

The following are an introduction of the complexity of a model and of the estimator, to better understand bias and variance.

In machine learning, the **complexity** of the model indicates the degrees of freedom of the model, that is the number of adjustable parameters comprised in it; it depends also on the chosen model function, for example, if it is linear, or not. The concept of complexity is correlated to the concept of the capacity

of the model, which indicates the number of parameters: the higher is the complexity of the model, the higher is the capacity.

In machine learning, an **estimator** is an equation that is used to estimate a quantity by observing the some data. It is needed in the implementation of the machine learning algorithms, since, without it, the model wouldn't be able to estimate its parameters.

Bias

From a statistical point of view, the **Bias** of an estimator is the difference between the expected value of the estimator $\hat{\theta}$ and the true value of the parameter that is estimated θ :

$$\text{Bias}(\hat{\theta}, \theta) = \mathbb{E}[\hat{\theta}] - \theta. \quad (2.2)$$

So it measures the expected deviation from the true value of the function or parameter [14].

Usually, as the complexity of the model increase, the bias decrease. Figure 2.1 shows bias graphically.

Bias has also a different interpretation if we apply it to the data: it can be seen as the quantity of inclination or prejudice towards an event. Bias indicates the strength of simplifying assumptions made by a model to make the target function easier to learn. If it is high there are more assumptions on the form of the target data; if it is low there are less assumptions. Making assumptions on data is useful to avoid learning irrelevant information.

Variance

From a probability point of view, **Variance** is the expectation of the squared deviation (difference) of a random variable X from its expected value:

$$\sigma_X^2 = \mathbb{E} \left[(X - \mathbb{E}[X])^2 \right]. \quad (2.3)$$

It provides a measure of the deviation from the expected estimator value that the sampling of the data causes [14].

In statistics, variance basically indicates the intensity of dispersion, meaning it is a measure of much a set of numbers is spread out with respect their average value. From a machine learning point of view, variance indicates how much the model will change if different training data were used. Variance can be seen as the quality of either differentiation, divergence or inconsistency of

different predictions using different training sets. With high variance, changing the training data would cause large changes of the estimate of the target function; at the same time, low variance would correspond to small changes of the estimate of the target function. Since the target function is estimated from the training data, it should adapt a bit to it and therefore it should have some variance. At the same time, it shouldn't have too much variance, otherwise, if the machine learning algorithm processes new data, the model would change too much, thus failing at picking the hidden underlying mapping between input and output.

Usually, as the complexity of the model increase, the variance between the model trained on different data increase. Figure 2.1 shows variance graphically.

The utopistic goal for a perfect machine learning algorithm is to have both low variance and bias. However, as a matter of fact, if we try to increase the model's capacity, bias and variance behave in an opposite way: the bias decrease and the variance increase. The actual goal is to find a good tradeoff between these two properties in such a way that the model reaches the correct complexity to learn the important information and discard the random noise. Figure 2.2 shows the tradeoff between bias and variance.

Overfit and Underfit

Bias and variance are correlated with the concepts of overfitting and underfitting.

- **Overfitting:** Overfitting can be defined as the production of an analysis that corresponds too closely or exactly to a particular set of data, and may therefore fail to fit additional data or predict future observations reliably [laxico]. In statistics, a model tends to overfit when it contains more parameters than needed to learn correctly the true distribution. Overfitting happens when the noise (residual variation) is accidentally extracted as if it contains some useful data that contributes to represent the underlying model structure. In machine learning, an overfitted model is a model having high variance and low bias on the training set, which causes poor generalization capabilities. When a model overfits, it learns too many characteristics of the training dataset, including the the useless ones, like noise or characteristics given by the incompleteness of the training data. Indeed, the training data should ideally represent and cover all the true

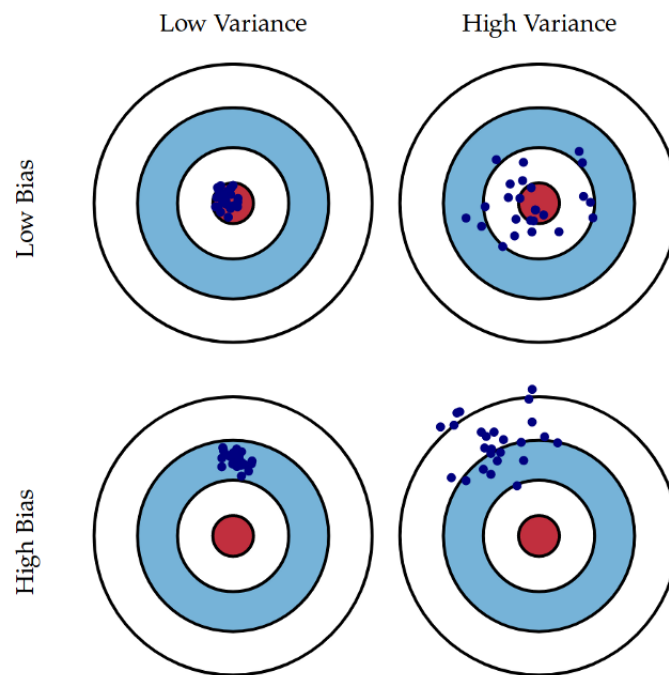


FIGURE 2.1: Illustration of bias and variance. The blue dots represent the learned distribution while the red circular target is the actual distribution. Since bias indicates the difference between the expected value and the true value, with high bias they are more distant, with low bias they are closer. Since the variance indicates the intensity of dispersion, with high variance points are more scattered, with low variance there are less scattered. In the image, is also considered the combination of bias and variance. There are indeed all 4 combinations: high bias and high variance, high bias and low variance, low bias and high variance, low bias and low variance. The best condition is low bias and low variance; in that case the model prediction coincides with the true distribution.

distribution in an accurate way, but this is not actually the case. Usually, training data has some lack at representing the true distribution. We can say that the model learns the training data too well. Figure 2.3 shows a graphical example of overfitting.

- **Underfitting:** Underfitting happens when the model can't sufficiently capture the underlying structure of the data. A model that underfits doesn't learn all the correct parameters to describe correctly the target distribution. In machine learning, an underfitted model is a model that is incapable to generalize due to its simple nature, which causes excessive generalization and a too simple model. The model is therefore not able

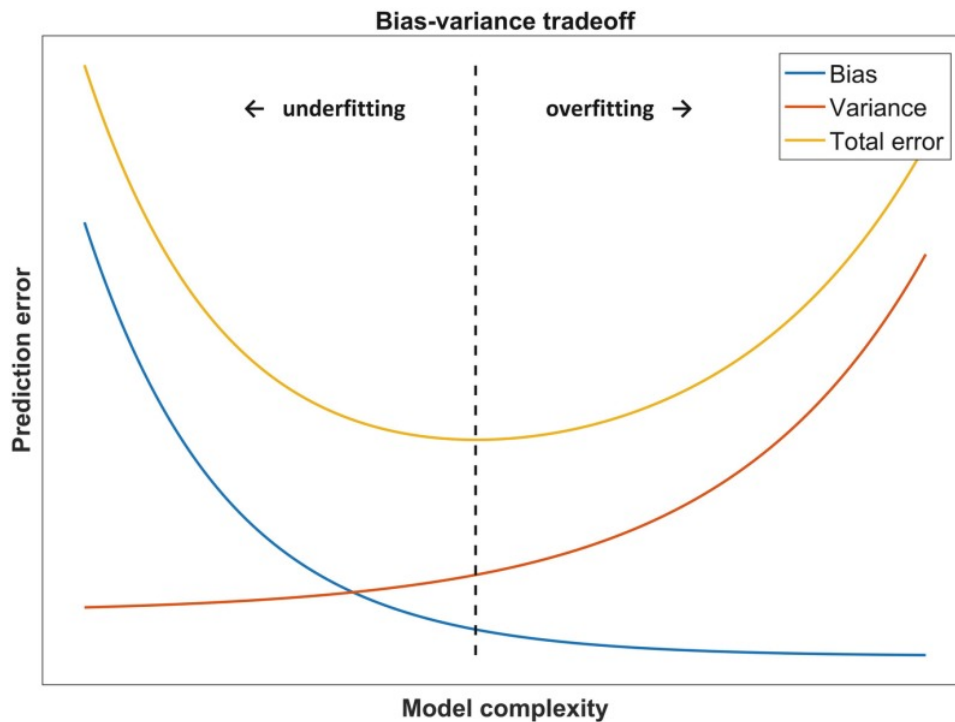


FIGURE 2.2: Illustration of the bias variance tradeoff and the correlation between overfitting/underfitting and bias/variance. As the bias decrease, the model complexity increase; as the variance increase, the model complexity increase. The yellow line indicates the total error as sum of the bias error and the variance error. The bias (variance) error is more specifically the prediction error given the influence of the bias (variance). The objective is to obtain the best model complexity to reduce the prediction error as much as possible. From the graph we can see that bias and variance behave in an opposite way and there is not a model complexity where both bias and variance are very low; instead, we have to find the best tradeoff that coincides with the lowest total error. We can also see that on the left, with high bias and low variance, we get underfitting; on the right, with low bias and high variance, we get overfitting.

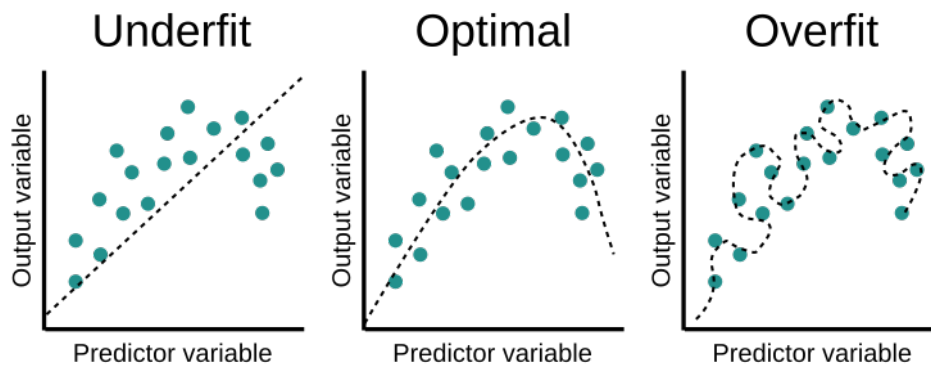


FIGURE 2.3: Illustration of overfitting and underfitting. The problem of underfitting as the result of trying to fit a linear model to nonlinear data can be seen in this example. Indeed, the model generalises too much, losing the ability of capture the shape of the distribution. On the other hand, overfitting happens when there is low generalization (high particularization). In this example the model captures too many characteristics, including noise (meaningless details), shaping itself too much to fit perfectly through all the points. In the middle we can see a good balancing between overfit and underfit, where the model doesn't generalize too much in order to avoid underestimating important properties and, at the same time, doesn't generalize too little in order to avoid overestimating useless properties.

to define the problem in all its complexity: it doesn't learn all the useful characteristics of the training dataset in order to represent correctly all the distribution peculiar shapes. We can say that the model learns insufficiently. For instance, underfitting happens when trying to fit a linear model to nonlinear data. Figure 2.3 shows a graphical example of underfitting.

Validation set

In summary, overfit comes when we pay too strict attention to the data; underfit comes when we ignore too much the data. So basically the goal is to find a good balance between attention and inattention in order to avoid overfitting and underfitting. A well-established solution in data science is called *validation*. As we previously said, we have two sets, the training set and the test set. Initially, the model learning is done through the training set, that is used to fit the parameters θ of the model, letting the model learn something about the data.

The test previously introduced is also used to check if the model has overfitting or underfitting problems. It can be assessed by looking at how well the model fits the train and the test; if it fits quite well both than minimal overfitting and underfitting has taken place. On the other hand, if there is a very good fitting on the train set and a quite worse fit on the test set it means that there is probably overfitting, since the model is not able to generalize enough to be flexible with new data.

The validation set is a bit similar to the test set. It is still used to evaluate the model performance but with the difference that the goal of taking countermeasures in order to improve the model. It can be seen as a "pre-test" to evaluate the model before the actual test set takes place. The test set, on the other hand, is used as a real world evaluation, that must be done at the very end as a final check of accuracy, sensitivity, specificity or F-measure. With just the the test set we can't know ahead of time how our model would behave with the final test set. The validation set therefore used to tune correctly the hyperparameters, that are all the parameters that characterise the architecture of the model. If the model's performance is not very good with the validation set, it means that the hyperparameters are not very good and therefore they need to be changed until the model behaves correctly. The validation must follow the same probability distribution as the test set and the training set [29].

Preventing Overfit and Underfit

With validation or test set we are able to detect overfitting or underfitting; this is useful but it doesn't solve them. Researches in this field have improved a lot and now there are several possible solutions that can be tried. In order to solve underfitting there are not a lot of options, the only solution is to just try alternative machine learning algorithm or increase the model's capacity/complexity. On the other hand, there are various ways to prevent overfitting. We will see some of them:

- **Cross-validation:** Cross-validation is one of the options to prevent overfitting and it is a quite powerful preventative set of tools. The training data is splitted into multiple smaller train-test batches. The use of these splits is to tune correctly the model. The one commonly used is the k-fold cross-validation, where the data is partitioned into k splits or *folds*. Then, during training, the algorithm is iteratively trained on all the folds except for just one that is used as the test set, that is the *holdout* fold. On

the next iteration, the holdout fold's role is assigned to another fold and all the remaining ones compose again the train test. The iterations stop when all the folds have been holdout folds for one time. Cross-validation needs less resources since it allows to tune hyperparameters of the model with just the original training set (there is no need of another new set for validation). Basically, the holdout fold is seen as an unseen dataset for the model. If we have a lot of data cross-validation is not necessary since a validation set can be extracted from the data.

- **Train with more data:** An attempt that can be done is to train with more data. It can help algorithms to detect better the underlying structure of the data, but it won't work every time. If the model is fed with a lot of training data, it won't be unable to overfit on all the variety of the samples and therefore it will have to do a more intense generalization in order to obtain good results. As a result, increasing the quantity of data will increase the accuracy of the model. However, this method is quite expensive (also compared to cross-validation) since collecting a lot of examples is an hard work and moreover the data used should be relevant and clean. This is need since, if there is more data but is more noisy, this technique won't work.
- **Data augmentation:** Another technique similar to the previous that is based on the concept of training with more data is the data augmentation, which is less expensive with respect to the former. If it is not possible to obtain new data, an idea is to modify the available data and make it appear diverse, increasing the variability. Data augmentation changes a data sample making it look slightly different. Also this methods helps the model to not learn useless characteristics since it is forced to generalize.
- **Data Simplification:** The previous methods like training with more data and data augmentation doesn't work all the time. Even if there is a large amount of examples, the model might still fail and overfit to the training set. Therefore, overfitting can happen due to the high complexity of the model. Data simplification approach consists in reducing the complexity of the model to make it so simple that it does not overfit. A data simplification operation can be for example reducing the number of parameters or the number of layers. Simplifying the model can also optimize the code and make it faster and lighter.

- **Regularization:** Regularization is a family of techniques applied to the model with the goal of limiting its chance of overfitting. the technique is chosen according to the learner used. For instance, the dropout and penalty parameter to the cost function are both regularization techniques that limit the complexity of the model. The regularization can depend on a bunch of parameters that are treated as hyperparameter too. These parameters can be adjusted using the validation set like any other hyperparameter.
- **Ensembling:** An ensemble is a machine learning technique that consists into combining the predictions of several distinct models. The most common methods are *boosting* and *bagging*. Boosting consists into combining very simple models, obtaining an ensemble with an increased complexity with respect to the single simple models. These weak learners are arranged in a sequence and the training proceeds in the following way: each model learns the outcome of the previous model. This means that each model learns how to correctly predict parts of data where the previous model was inaccurate. It's like each model focuses in a portion and becomes (ideally) a good predictor in that portion. So boosting is the combination of weak models in the attempt of obtaining a stronger one. Bagging, on the other hand, consists in combining strong independent models organized in parallel with the goal of improving the prediction. More specifically, the objective is to reduce the overfitting odds of complex models.
- **Early Stopping:** During training, as we have seen, initially the training gives its benefits and improve the performance of the model. However, after some iterations, it starts to overfit the training data, worsening its generalization performance. Therefore, a possible solution to avoid overfitting is to stop the learning process slightly before the model starts to overfit. This method is called early stopping and it is performed by periodically checking on the validation set.

2.2 Deep Learning

Deep Learning or Hierarchical Learning is a subset of Machine Learning in Artificial Intelligence that uses multiple layers to progressively extract higher-level features from the raw input [10]. It can imitate the data processing function of the human brain and create similar patterns the brain used for decision making [kechit2019deep].

At the core of the Deep Learning algorithms there are the Artificial Neural Networks that is biologically inspired by the human brain. The Neural Network is a network of neurons interconnected together that can communicate through signals between each other. Thank to the interaction between neurons, an emergent intelligent behaviour can appear: even if a single neuron is not intelligent, a system of them can be. Neural networks allows computer programs to recognize patterns and solve common problems [5].

The unit of a Artificial Neural Network is the perceptron, the artificial neuron. A perceptron is a mathematical function that simulates the biological neurons.

The human neuron works in this way: it takes input signal (coming from other neurons) through dendrites; the signals are weighted separately through synapses (synapses can be stronger or weaker); then the signals are summed up together; finally, if the total signal exceeds the neuron threshold, the signal is sent, otherwise not. The perceptron works exactly in the same way. The perceptron is composed of inputs, weights, output, bias and activation function. There is a tight correspondence between the perceptron and the biological neuron: perceptron inputs corresponds to neuron dendrites, output corresponds to axon, weights corresponds to synapses. The bias and activation function in a perceptron are used to simulate the switch of the biological neuron: if it is *on* it sends signal, otherwise not. The mathematical expression of the perceptron is the following:

$$y = f \left(\sum_i w_i x_i + b \right), \quad (2.4)$$

Figure 2.4 shows a graphical schema of the perceptron.

There are several activation functions, that is a function that tells if the neuron is activated or not depending on the value of the summation of the

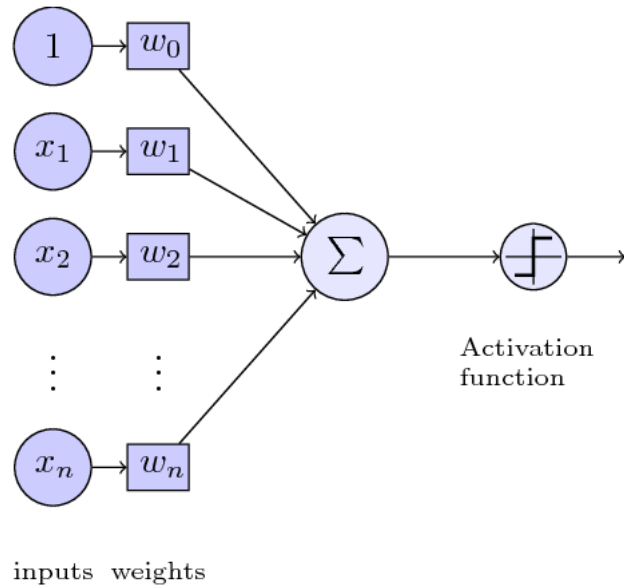


FIGURE 2.4: Illustration of a Perceptron: it takes the inputs, weighs them separately, sums them up and passes this sum through a nonlinear function to produce the output.

weighted inputs. An activation function is a nonlinear function and the non-linearity is needed because otherwise the NN reduces to a single logistic regression. The most popular activation functions are the ones shown in Figure 2.5. More specifically:

- **Sigmoid:** If x is the resulting summation of the weighted inputs, the function is the following:

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \quad (2.5)$$

This function has two problems. Firstly, the output is not centered in 0 (in sigmoid it is actually in 0.5). This is a problem since, from an intuitive point of view, the perceptron should activate when the input is higher than 0. However, with sigmoid, when input is 0 the perceptron is already half activated, indeed we get 0.5 (full activation corresponds to 1). Secondly, the function saturates with very high values; indeed, if the input value is higher than 5 (roughly) the outcome will always be 1, independently from the value. This is a problem because there wouldn't be a gradient and therefore the gradient descent wouldn't work 2.16.

- **Tanh:** If x is the resulting summation of the weighted inputs, the function is the following:

$$\tanh(x) = 2\sigma(x) - 1, \quad (2.6)$$

This activation function is better than the previous one since it is centered in 0. However, it has still the saturation problem.

- **ReLU:** If x is the resulting summation of the weighted inputs, the function is the following:

$$f(x) = \max(0, x), \quad (2.7)$$

The ReLU activation function is a non linear function. It is better with respect to the Tanh since it doesn't have the saturation problem; for a given input greater than 0 the output always changes. This function has also the advantage to be computationally faster than the previous ones.

- **Leaky ReLU:** If x is the resulting summation of the weighted inputs, the function is the following:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ ax & \text{if } x \leq 0 \end{cases}, \quad (2.8)$$

If we look closely, the saturation problem is also present with input lower than 0; indeed, the derivative of the negative part is 0 causing the model to not learn. The Leaky ReLU solves this problem, adding a little slope to allow the gradients to flow on during training. In the case of relu function, weights should be low near 0 since it doesn't squash values like sigmoid, otherwise they might grow too much over time.

If we interconnect several perceptrons together we get a Multi Layer Perceptron (MLP) or ANN. A Feed Forward NN is a ANN that is comprised of several layers of perceptrons; to be more precise, there is always an input layer that takes the features as input, and output layer that gives classes as output (if we consider a classification problem) and one or more hidden layers between the input and output ones. The number of input perceptrons (in the input layer) coincide with the number of features and the number of output perceptrons coincide with the number of classes or properties. Each perceptron of a layer is connected to all the other perceptrons in the adjacent layers: the current perceptron takes as inputs the outputs of the previous layer and, at the same time, its output is given in input to each perceptron of the next layer. The universal approximator theorem states that a MLP with just one hidden

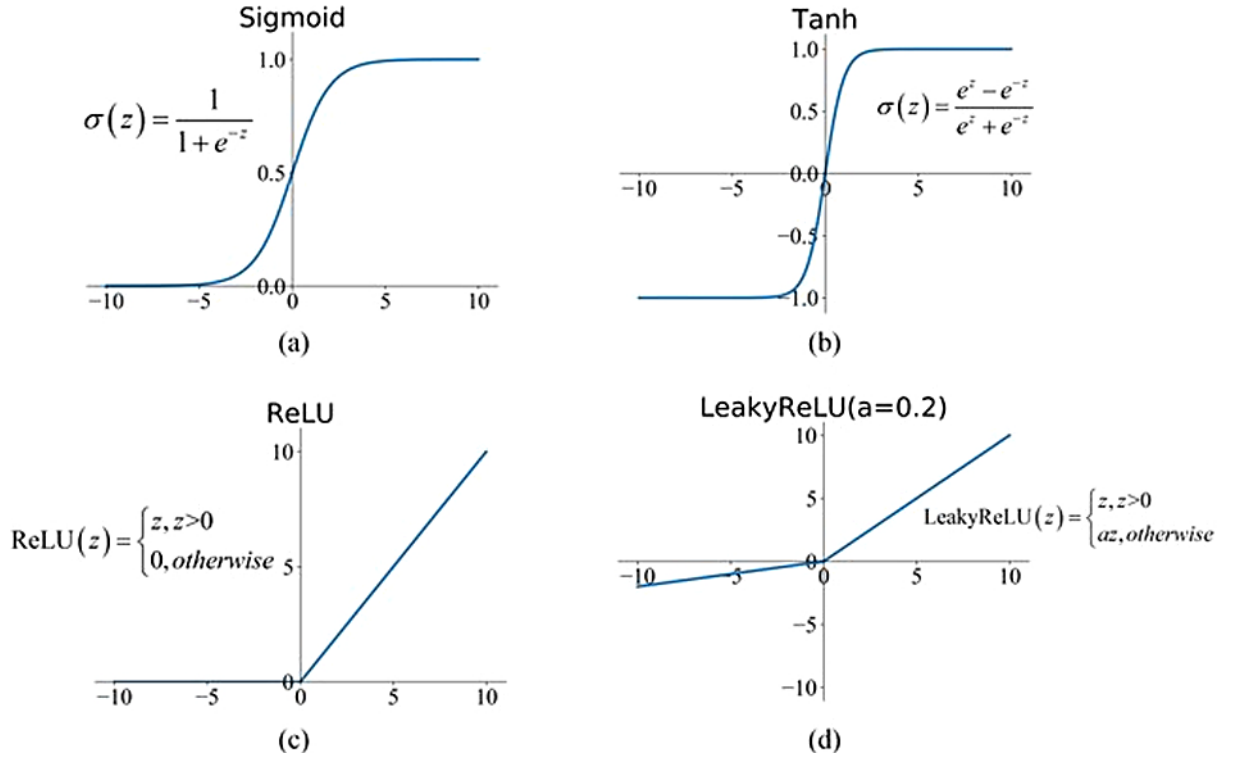


FIGURE 2.5: Cartesian graphs of the activation function with the corresponding formulas next to it.

layer can approximate any possible continuous function. Figure 2.6 shows an example of feed forward network.

In order to control the learning process we have to modify some parameters that are called hyperparameters. In the case of an ANN hyperparameters are: the number of layers, the number of perceptrons for each layer and the activation function used. The capacity of a network, that is determined by the number of layers and neurons, must be tuned since if it is high it is more representable and is able to model more complex functions, but, at the same time, it needs more training and it is more prone to overfit. So, the best choice is to find a good tradeoff between these phenomena. Usually hyperparameters are set to reach the computational budget. The problem of overfitting can be overcome with regularization techniques.

As always, in every machine learning algorithm there is the training phase and this happens also for the ANN. The training consists in minimizing the objective/loss function:

$$L(\theta; \mathbf{x}, \mathbf{y}) = \frac{1}{N} \sum_i L_i(\theta; x_i, y_i) + \lambda \Omega(\theta), \quad (2.9)$$

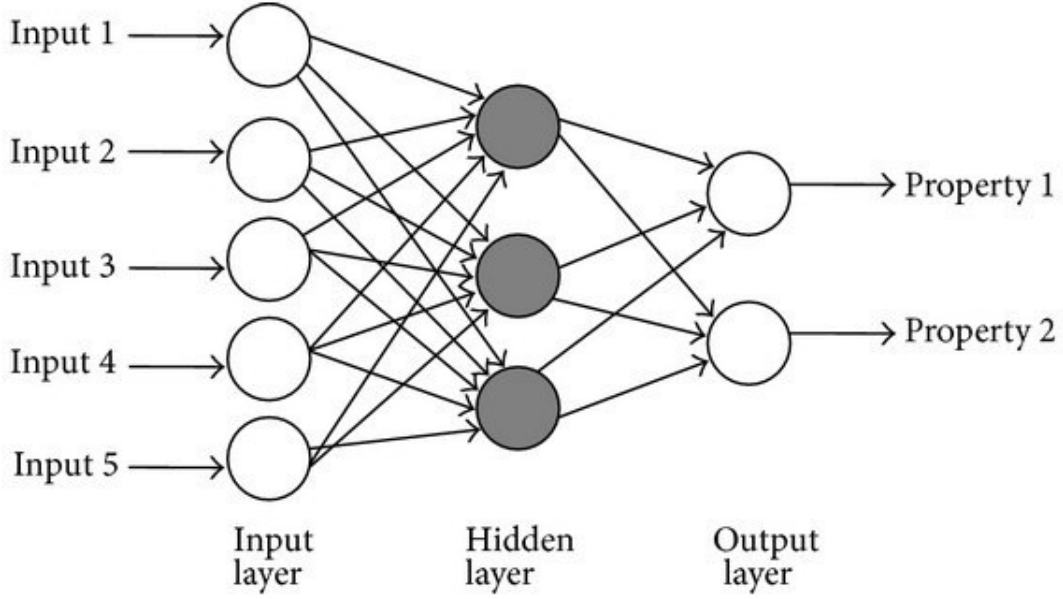


FIGURE 2.6: A multi layer perceptron with one hidden layer. The input, the hidden and the output layers are composed of 5, 3 and 2 perceptron respectively.

where N is the number of instances, L_i is the task loss for each instance, θ are the weight and bias parameters of the perceptron expression, λ role is to calibrate the regularization term and Ω measures the quality of mapping between the input and the output. Different objective functions L_i that can be used depending on the tasks; for example:

1. Classification:

(a) **Hinge Loss:** The formula of Hinge Loss is the following:

$$L_i = \sum_{j \neq y_i} \max(0, f_j - f_{y_i} + 1), \quad (2.10)$$

In the expression f_{y_i} is the correct score while f_j is score of all the other classes. This loss forces classes to be distant from the correct one of a fixed amount. The hinge loss is used for maximum margin classification, like support vector machines.

(b) **Softmax Loss:** The formula of Softmax Loss is the following:

$$L_i = -\log \left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}} \right), \quad (2.11)$$

Softmax Loss can be obtained basically from the combination of the Softmax Activation and the Cross-Entropy Loss. Cross Entropy loss is the sum of the negative logarithm of the probabilities.

- (c) **Binary Cross Entropy:** The cross entropy between probability distributions p and q indicates the number of bits needed to encode a sample of p using a code optimized for q :

$$H(P, Q) = - \sum_{x \in \mathcal{X}} p(x) \log(q(x)), \quad (2.12)$$

where $\mathcal{X}$ is a probability space where p and q are defined. If we consider p as the real distribution and q as the one predicted by the model, the Binary Cross Entropy Loss can be computed as follows (the summation over N is already in 2.9 formula):

$$L_i = - [x_i \log(q(x_i)) + (1 - x_i) \log(1 - q(x_i))], \quad (2.13)$$

where i is the index of the N samples, x_i is the actual class value (0 or 1) of i , $q(x_i)$ is the probability estimated by the model that the i -th sample is x_i [12].

2. Regression:

- (a) **Mean Squared Error:** The formula of Mean Squared Error is the following:

$$L_i = ||f - y_i||_2^2, \quad (2.14)$$

The mean squared error indicates the proximity of the function to a set of points. It is obtained by computing the square of the distances from the points to the regression line (distances can also be called errors and this is why it is called the mean squared error). The squaring is done to remove negative signs and to weight more the larger differences.

Neural Network Training

The training consists in find the set of parameters that minimize the loss function:

$$\theta^* = \operatorname{argmin}_{\theta} \left(\frac{1}{N} L(\theta; x, y) \right), \quad (2.15)$$

From a more practical point of view the training is done as follows: initially it starts with a random set of parameters; then, the gradient of the loss function is computed with respect to each parameter (this tells us how much the parameters need to be altered); finally the parameters are updated. This sequence of operations are executed for several training instances till convergence is reached.

The procedure of updating the weights in such a way that the loss function is minimized it's called **Gradient Descent**. The standard Gradient Descent method updates the parameter w in the following way:

$$w_{j+1} = w_j - \frac{\eta}{n} \sum_{i=1}^n \nabla Q_i(w_j), \quad (2.16)$$

where w_{j+1} indicates the next weight, w_j the current one, η is the learning rate that adjusts the step size at each iteration, Q_i is the function associated to the i -th observation of the training set of n samples and ∇Q_i is the vector of partial derivatives of Q_i that indicates the direction of maximum change of Q_i . The standard Gradient Descent is quite accurate since the gradient is computed as an average of the gradient of the function for all the instances, but it is computationally expensive. A more economic version is the **Stochastic Gradient Descent** (SGD) that it's the same as the standard one but with just one instance Q_i , so it's like it is an approximation. The SGD expression is the following:

$$w_{j+1} = w_j - \eta \nabla Q_i(w_j). \quad (2.17)$$

So, in order to update the parameters we need the gradient of the loss function with respect to each parameter. The gradient is computed through **back-propagation**. Back-propagation is based on the chain rule principle, that can be defined in the following way:

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx} \quad \text{where} \quad z = f(g(x)), \quad y = g(x), \quad (2.18)$$

The back-propagation is needed to change the local process of gradient descent, executed by each neuron, to a global one, since neurons don't know the topology of the network (they don't know the structure of the network, the number of layers, the number of neurons or which neuron is before or after them). Summarizing, each neuron does two basic operations, that are the following:

$$\frac{\partial h_{i+1}}{\partial W_{i+1}} \frac{\partial h_{i+1}}{\partial h_i}, \quad (2.19)$$

In the previous expression h_{i+1} is the output while W_{i+1} is the output's weight and h_i is the input; all of them are derived.

Figure 2.7 [41] shows how back-propagation works. The process in the figure can be explained in the following way. For each instance/example (x) a forward propagation is executed, computing and obtaining at the end the value of the loss function (L). Then, the derivative of the loss function is computed with respect to parameters of each neuron starting from the last layer ($\partial L / \partial o$), proceeding to the penultimate ($\partial L / \partial h_2$) and so on. From this step comes the name back-propagation since it is a chain rule that is computed backward in the network. At the end of this process, each neuron obtains the derivative of the loss respect its parameter/weight ($\partial L / \partial W$). In the back-propagation process we have also to consider the derivative of the activation function.

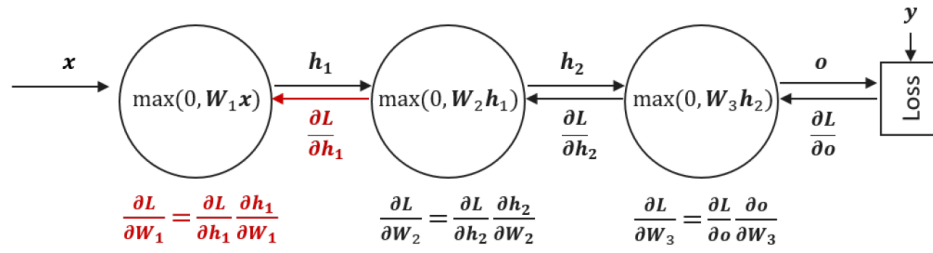


FIGURE 2.7: Schema of back-propagation. Each circle is a layer, x is the input instance, y is its corresponding class, L is the loss, h is the output of one neuron, W is the parameter. The arrows that goes from the left to the right are the several steps of the forward propagation. On the other hand, the arrows that goes from right to left are the steps of back propagation. The operations of back-propagation follows the chain rule principle.

To be more precise, for each neuron we have the following: the current neuron obtains the derivative of the loss with respect to its output from the next neuron. Then, this quantity is multiplied with the derivative of the output with respect to the input obtaining the derivative of the loss with respect to the input 2.20. This quantity will be back propagated to the preceding neuron that, from its prospective is the loss with respect to its input; the cycle restart. The complete expression that each neuron has to compute during back-propagation is the following:

$$\frac{\partial L}{\partial \mathbf{h}_i} = \frac{\partial L}{\partial \mathbf{h}_{i+1}} \frac{\partial \mathbf{h}_{i+1}}{\partial \mathbf{h}_i}, \quad (2.20)$$

Thank to that computation, each neuron will gain the knowledge of the contribution on the whole network, that is the derivative of the loss with respect to its output. This quantity is then multiplied with the derivative of the output with respect to the parameters obtaining the derivative of the loss with respect to the parameters:

$$\frac{\partial L}{\partial \mathbf{W}_{i+1}} = \frac{\partial L}{\partial \mathbf{h}_{i+1}} \frac{\partial \mathbf{h}_{i+1}}{\partial \mathbf{W}_{i+1}}, \quad (2.21)$$

This resulting quantity is basically the influence of the neuron parameters on the final output. Thank to this information each neuron is able to understand how to change its parameters.

There is one important thing to consider: the weights need to be randomly initialized. If this condition is not met and all the weights are initialized with same number, a symmetry is generated, leading to a symmetric behaviour of the neurons that start to behave all in an identical way. If this happens the network won't work properly.

Artificial neural networks with a lot of layers and neurons have a very high representational capacity. This means that they are prone to overfit. We should avoid overfit and to do that we need regularization techniques. We have already seen some of them in a generic context; now we will observe them from a neural networks point of view:

- **Dropout:** Each time training is done, only a random subset of neurons are activated while the others are dropped. It is like a sub-network of the whole network is trained. On the other hand, during the testing all the neurons are activated. During test time, it's like all the sub-networks are working at the same time, therefore the final global prediction is the average of the prediction of each single sub-network. So, it can be seen as an ensemble of sub-networks.
- **Data augmentation:** It is useful when there training data is small but at the same time we want the model to reach a good generalization capacity. Usually, more data there is, more generalization capability the model gets; Ideally, we would need more data. In order to increase the data we can generate new training instances by applying transformations to original instances. This procedure is called data augmentation. Data

augmentation can be useful also to reinforce the model to recognise the correct class even if some instances are distorted, so that the model can learn to neglect the noisy information (for instance, it can be done for images).

- **Early stopping:** The early stopping is a method that consists into stopping the training when the performance on validation set (that in this case is used as a test set) starts to get worse. There is also a variation in which the training time is protracted more and the learning process stops when the validation loss is already stable or start to increase.
- **Ln regularization technique:** In the objective function of the generic ANN there are several terms and one of them is $\Omega(\theta)$ that is the regularization function. This function can be any norm; there are several norms and among them there is also L1 and L2. L1 can be expressed as the sum of the absolute of all parameters:

$$\sum_i |\theta_i|, \quad (2.22)$$

If we use the L1 regularization function in the loss function, the loss function derivative becomes basically the sum of all the non-zero parameters. Since the objective is to minimize the loss function, all the involved parameters are also minimized. If the parameters are minimized to zero it's like they are turned off. This leads to less complexity and therefore also to less overfit. There is also L2 that is basically a smoother version of L1.

$$\sum_i (\theta_i)^2, \quad (2.23)$$

In the case we use L2 regularization, the loss becomes:

$$L_{final} = L + \sum_i (\theta_i)^2, \quad (2.24)$$

where λ is the *weight decay* that is used as scale factor to compensate the L2 contribution. If we consider the resulting expression of the Stochastic Gradient Descent, the weight w update becomes the following:

$$w_{i+1} = w_i - \eta \frac{dL}{dw_i} - 2\eta\lambda w_i, \quad (2.25)$$

where i is the index of iteration ($i + 1$ means "next iteration"), $\frac{dL}{dw_i}$ is the gradient and η is the learning rate. This expression is obtained computing the Stochastic Gradient Descent 2.17 of function 2.24.

To sum up, learning is the process of adjusting the weights through gradient descent allows the model to understand the direction to take to improve performance and reduce the errors. Each time there is a new sample during training the back-propagation is executed improving the model parameters until it converge at the minimum.

There are several types of ANN. Here there are some of them [5]:

- **Perceptron:** The perceptron is the oldest and most basic NN, devised by Rosenblatt [37]. It is composed by just one single neuron.
- **Feedforward neural network:** It is the standard neural network with an input layer, one or more hidden layer and an output layer. These networks are comprised of sigmoid neurons since most real-world problems are nonlinear. Most of neural networks like vision or natural language ones comes from this network.
- **CNN:** Convolutional neural networks have a lot in common with feed-forward networks. They are usually applied on computer vision (image/pattern recognition). To be more precise, they are useful when some of the information of the data comes from the relative position of instances. Computations in this network are usually matrix multiplications, that is the convolutional operation at the core of this network.
- **RNNs:** Recurrent neural networks are algorithms primarily used for time-series data analysis to make predictions on future outcomes, for example stock market predictions.

Autoencoders

Autoencoders are feed-forward networks that are trained to copy its input x to its output r passing through hidden code h . It is composed by 2 parts:

- **encoder:** reduces information from the input to the hidden code.
- **decoder:** reconstructs information from the hidden code to the output.

The goal is to extract frequent patterns contained in the input (typical of unsupervised learning). The model learns by comparing the input and the output: the more similar they are, the better the information found is. It's like autoencoders compress the data and then decompress it; frequent patterns are better compressed than uncommon ones.

GAN

A **Generative Adversarial Network (GAN)** is a class of deep learning frameworks designed by Goodfellow et al. [15]. Though originally proposed as a form of generative model for unsupervised learning, GANs have also proven useful for semi-supervised learning, fully supervised learning, and reinforcement learning.

The goal of GANs is to create new examples similar to the ones in the training set. Indeed, GANs are **Generative Models** and consist in learning the probability density function of how elements are distributed in the training dataset. Therefore, after training, new elements can be sampled (generated) from this function.

By comparison, autoencoders are discriminative: they try to learn a predictor given the observation $P(Y|X=x)$ without probabilistic interpretation of the code so it's difficult to sample new elements. However, generative models can. They try to simulate how data is generated in order to understand underlying causal relations $P(X|Y=y)$ learning the probability.

GANs consist in 2 networks that compete with each other: the generator and the discriminator. To be more precise:

- **Generator:** its role is to create new samples, that are sampled from its own data distribution. The samples generated by the generator's distribution are considered the fake data, in contrast with the training distribution and its samples that are considered, on the other hand, the true data.
- **Discriminator:** it is a 2 class classifier that takes in input samples coming from the fake generator's distribution and the true training distribution. The discriminator's role is to distinguish if the sample is a fake or a true one.

These 2 networks compete with each other. The objective of the generator is to fool the discriminator, so it has to learn a distribution such that the samples are similar to the training ones. To be more precise, the generator learns

to map from a latent space to its own data distribution (so it takes a random input). On the other hand, the objective of the discriminator is to not be fooled by the generator, so it has to learn how to distinguish correctly between the generator's samples and the training samples. In other words, the discriminator has to minimize its own error rate while the generator has to maximise the discriminator's error rate. The two neural networks objectives are in contrast with each other in the form of a zero-sum game, where one agent's gain is another agent's loss. Figure 2.8 shows the schema of a GAN model.

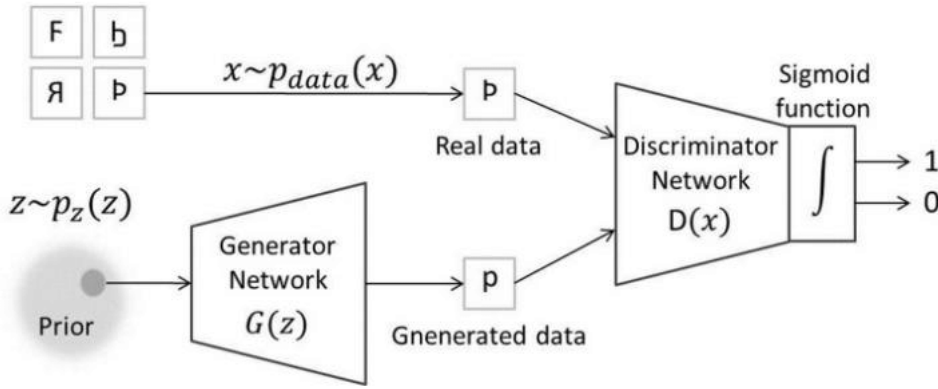


FIGURE 2.8: Schema of the GAN model [47]. The schema shows the interaction between the generator and the discriminator. The discriminator takes in input one instance of the real data or one instance of the generated data. Then, it computes if it is real (1) or generated (0). The generator generates samples taking in input random values from a uniform distribution ("prior" in the schema). In this case, the data are a bunch of symbols.

The **loss function** that express this competition is the following:

$$\min_G \max_D V(G, D) = \mathbb{E}_{x \sim P_{data}(x)} \log D(x) + \mathbb{E}_{z \sim P_z(z)} \log(1 - D(G(z))), \quad (2.26)$$

As we can see, on the right of the equation, the first addendum is the log probability of the discriminator predicting that the real-world (training) data is genuine; while, the second addendum is the log probability of the discriminator predicting that the generator's generated data is not genuine. The discriminator has to maximise these probabilities, so it has to perform gradient ascent. In contrast, the generator has to minimize these probabilities by a gradient descending.

During training, thanks to this generator-discriminator competition, both networks improve. The discriminator gets better at discriminate the real and fake samples, forcing the generator to generate samples even more similar to the real ones. The generator, on the contrary, gets better at generate samples similar to the real ones, forcing the discriminator to get even better at discriminating the real and the fake samples. If the training proceeds correctly, both discriminator and generator will improve to a point where the real and fake samples are almost indistinguishable. This means that the generator has learned a distribution very similar to the training one, so it's like it has learned something about real data. The two networks are trained separately alternately with two independent backpropagation procedures.

In game theory, the GAN model converges when both the discriminator's loss and the generator's loss reach the **Nash equilibrium**, that is when one player will not change its action regardless of what the opponent may do. Figure 2.9 shows generator and discriminator loss.

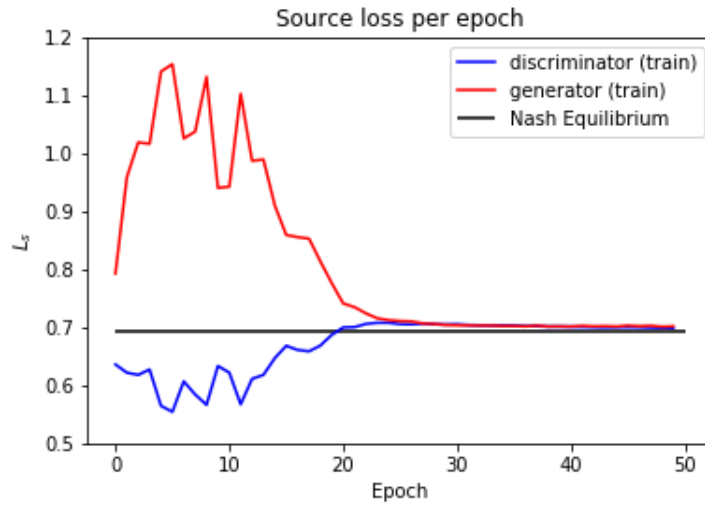


FIGURE 2.9: Schema that shows the correct behaviour of generator and discriminator losses [23]. They both reaches the Nash equilibrium and then the losses doesn't change very much.

As the generator improves with training, the discriminator performance gets worse because the discriminator can't easily tell the difference between real samples and fake samples. If the generator succeeds perfectly, then the discriminator will reach a 50% accuracy, obtaining the following:

$$D(x) \approx 1 - D(G(z)) \approx 0.5, \quad (2.27)$$

Initially, the quantity of $D(x)$ is 0 while $1 - D(G(z))$ is 1. If the training proceeds correctly, the $1 - D(G(z))$ should decrease while $D(x)$ increase till reaching both 0.5. It means that the discriminator behaves randomly since it is not able to distinguish the real samples from the fake ones. After reaching the value of 0.5 the training should stop. Indeed, if the GAN continues training past the point when the discriminator is giving completely random feedback, then the generator starts to train on junk feedback, without learning anything. Figure 2.10 shows what happens to the two quantities ($D(x)$ and $1 - D(G(z))$) during training.

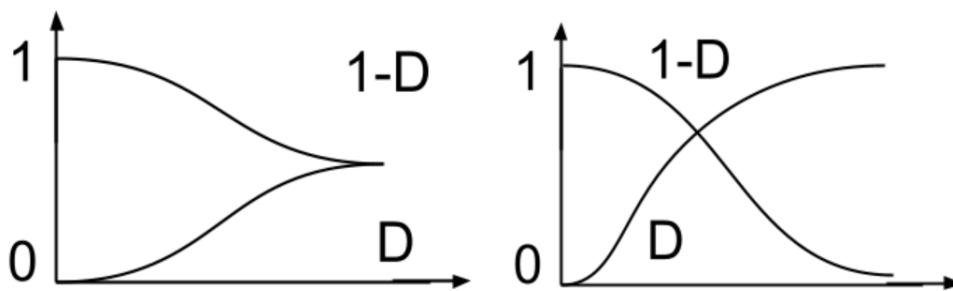


FIGURE 2.10: Schema that shows $D(x)$ and $1 - D(G(z))$ quantities of the GAN loss along time during training. The abscissa indicates the time while the ordinate the values of the two quantities. Initially $1 - D(G(z))$ value start to 1 while $D(x)$ value start to 0. As the training proceeds, if nothing go wrong, the two quantities gets closer till reaching 0.5 (left graph). Otherwise, if there is not a balance between generator and discriminator, the two quantities would cross each other and then step away (right graph).

Algorithm 0 shows the specific steps to train GANs, from Goodfellow et al. [16].

GANs are quite powerful and are used in several fields. However, they have a big problem. Since GANs are based on a **zero-sum non-cooperative game**, the training is not trivial since both generator and discriminator improves only if a balanced competitive state is established. Their improvement works thanks to a **delicate balancing** between their performance. If one of the two performs much better than the other the balancing breaks, bringing the system into an unbalanced competition where no one improves: the weakest has no room for error, since the other is too strong, preventing it to improve; while, the strongest is not challenged enough to improve, since the other is too weak. For example: if the discriminator is too good, it will return values so close to 0 or 1 that the generator will struggle to read the gradient; if the generator is too good, it will persistently exploit weaknesses in the discriminator

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyper-parameter.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

that lead to false negatives. In order to obtain a balanced situation, a correct choice of hyper-parameters is needed, like the initialization of the weights, the number of layers and so on. The correct choice of hyper-parameters is not trivial.

This unbalancing problem can cause some of the well known problems [19][17]:

- **Non-convergence:** the model parameters oscillate, destabilize and never converge. If the unbalanced competition problem persists, the behaviour of $D(x)$ and $1-D(G(z))$ doesn't converge to 0.5 (as we have seen before) but the two quantities would progress into crossing and then into straying from each other, compromising the convergence. An attempt to remedy can be to add noise to discriminator inputs or to penalize the discriminator weights through regularization.
- **Mode collapse:** it happens when the generator learns a subset of the real distribution, it creates a limited varieties of samples that get stuck on it

and doesn't cover all the possible samples. This could happen because sometimes, the generator, in the effort of fooling the discriminator, starts to learn a subset of samples that can easily deceive the discriminator. If the generator starts producing a small set of outputs over and over again, the discriminator's best strategy is to learn to always reject that output. This brings the generator to change its distribution into another subset of the real distribution, succeeding to fool again the new discriminator. It can also happen that the discriminator gets stuck in a local minimum, facilitating the generator even more. So what happens is that, each iteration the generator over-optimizes for a particular discriminator, and the discriminator over-optimizes for a particular generator. Both networks are overfitting to exploit short-term opponent weakness. This turns into a cat-and-mouse game and the model will not converge. An attempt to remedy can be the Wasserstein loss or the Unrolled GANs, as we will see in chapter 3.

- **Diminished gradient:** it happens when the discriminator gets too successful that the generator gradient vanishes and learns nothing. An attempt to remedy can be the Wasserstein loss or the modification of the minmax loss.
- **sensitive hyperparameter:** The model can be highly sensitive to the hyperparameter selections, making it hard to tune it correctly. No cost functions will work without spending an impractical amount of time on the hyperparameter tuning.
- **general ML problems:** GANs can also have all the other common problems of machine learning algorithms like underfitting and overfitting.

3 Related Works

In this chapter we will focus on the **Mode Collapse** problem, the **Catastrophic Forgetting** problem and the **Continual Learning** field.

3.1 Avoiding mode collapse

Mode collapse is a problem that occurs when the generator learns to map several different input z values to the same output point.

— Goodfellow [13]

As we have seen in the previous chapter, one of the most common problems of the Generative Adversarial networks is **Mode Collapse**. In that case, the generator acquires just a portion of the whole distribution, therefore it creates a limited variety of samples that don't cover all the possible samples. This happens because the discriminator focuses on the small portion of samples created by the generator, learning to always reject them. The generator can then simply change the portion of the distribution to deceive the discriminator that is too focused on the previous portion. Usually, this effect keeps going, leading to a cat-and-mouse game where the model is not able to converge. The problem is that both networks overfit a portion of data, in particular the discriminator that is not able to observe the whole distribution and waste a lot of time to get out of its current focus.

In order to identify mode collapse, we can look at the two following events, if they happen the GAN can suffer from mode collapse (Brownlee [7]):

- The loss for both the generator and the discriminator oscillates over time.
- The generator network generates identical outputs from different input values coming from the latent space.

Mode collapse is a problem that emerges from the fragile balance that is needed between the generator and the discriminator. Indeed, the generator and the discriminator models are trained concurrently in a zero-sum game, where the profit of one network comes at the cost of the other one. The training

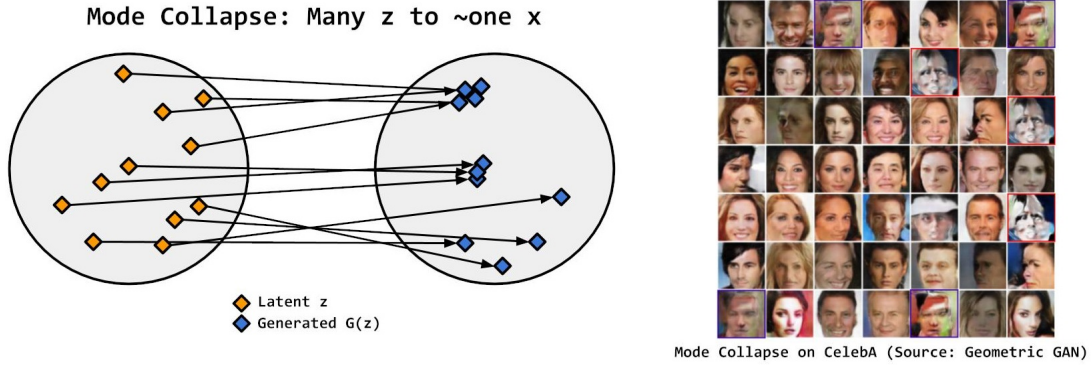


FIGURE 3.1: Illustration of mode collapse [1]. The schema on the left shows the problem of mode collapse from the point of view of the generator. The generator learns a mapping between the latent space and the output distribution that correlates different and scattered values of the latent space into very similar and close values in the output distribution. Basically, the generator proposes the same output instances multiple times. On the right, we can see a convolutional GAN applied on human faces. If we review the sample of generated images on the right, the images show low diversity. The generator proposes the same identical image or very similar identical images having basically the same face, repeated numerous times.

is successful only if equilibrium between the two models is reached. The two models are updated alternatively; this means that every time the weights of one of the two models are updated, the behavior of the optimization problem changes, conducting to the creation of a dynamic system. This increases the chance of non-convergence of the generator and discriminator, and also cause the mode collapse problem.

Mode collapse emerges also from the fact that the GAN model is limited by the fact that it works properly only if it is trained on just single-modal data. Single-modal data is data whose dependent variable consists of only one categorical entry [2]. With single-model data, there is just one type of data. Indeed, on the contrary, if we train the GAN on **multi-modal data**, it doesn't work properly, falling into the problem of mode collapse. An example of a multi-modal dataset is the dataset of pets where each mode is an animal species (a single-modal dataset is a dataset with, for instance, only cats inside).

Mode collapse can be limited with the following measures [2]:

- **Grouping the classes:** One method is to split the data into the different classes in the dataset, generating one group for each class. Thanks to this, the discriminator is able to discriminate for each different class.

- **Anticipating Counter-actions:** The goal of this method is to stop the mouse-cat game between the generator and discriminator by training the generator such that it is able to take into account the discriminator's counter-actions to maximally fool it. The negative aspects of this method are a longer training time and more complex gradient computation.
- **Learning from Experience:** This approach consists to re-propose to the discriminator old fake examples previously generated by the generator, in a fixed number of iterations. This method contemplates the presence of memory to store old examples. This memory can be seen as the bag of experience of the discriminator.
- **Multiple Networks:** This method consists of the use of multiple generators, each dedicated to a different class, covering all the classes. The negative aspects of this method are longer training time and worse fake data quality.

Several studies have been made about mode collapse and two possible remedies are the Wasserstein GAN or the Unrolled GAN. We will see Wasserstein GAN in detail.

Wasserstein GAN

The **Wasserstein GAN** [3] is an improvement of the vanilla GAN that is more stable and resistant to mode collapse. The main difference with respect to the vanilla GAN is that its loss is substituted with the Wasserstein loss.

The Wasserstein loss is based on the Wasserstein distance – also called Earth Mover distance –, which is a distance function that measures the distance between two probability distributions on a specific metric space M . Let's consider two distributions as two different ways of piling some dirt (earth) on a metric space M . If we want to move the dirt of one pile in the same place and arrangement of the dirt in the second pile, we can move them in several ways, so we have several possible transporting plans; each transporting plan has a cost. We can informally define the Wasserstein distance as the cost of the cheapest transporting plan between the dirt of one pile to the dirt of the other pile. The cost is computed as the quantity of dirt moved multiplied by the distance of their movement.

So, the Wasserstein distance is the minimum cost of converting a distribution q to another distribution p . In GANs, the Wasserstein distance between

the real distribution P_r and the fake (generated) distribution P_g is formally defined as the greatest lower bound for any transport plan, that is basically the cost for the cheapest plan (Hui [19]):

$$W(\mathbb{P}_r, \mathbb{P}_g) = \inf_{\gamma \in \Pi(\mathbb{P}_r, \mathbb{P}_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|], \quad (3.1)$$

$\Pi(P_r, P_g)$ denotes the set of all joint distributions $\gamma(x, y)$ whose marginals are respectively P_r and P_g (Arjovsky, Chintala, and Bottou [3]). Π is the group of all the possible transport plans γ .

Wasserstein GAN uses a new cost function inspired by the Wasserstein distance to improve the vanilla GAN. Vanilla GAN uses the KL-Divergence and the JS-Divergence to compute the distance between the real and fake distributions. If the generator's performance is not very good, the fake distribution q will be quite distant from the real one p . If this happens, the divergence between the distributions will increase and, consequently, its gradient will decrease going near zero. This leads to a generator that is not able to learn from gradient descent. Goodfellow, the author of vanilla GAN, proposed some solutions to counter this problem, but the outcome of the GAN still depends on the generator's performance.

Wasserstein GAN is better than the vanilla GAN since the distance function used leads to a smoother gradient everywhere, helping the generator to learn better, solving the problems of the original GAN. Figure 3.2 shows the gradient difference between WGAN and vanilla GAN.

The Wasserstein distance is quite good but, from a practical point of view, not implementable. So we have to modify it. The simplification is done thanks to the Kantorovich-Rubinstein duality, obtaining the following:

$$W(\mathbb{P}_r, \mathbb{P}_\theta) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim \mathbb{P}_r} [f(x)] - \mathbb{E}_{x \sim \mathbb{P}_\theta} [f(x)], \quad (3.2)$$

$\sup$ is the least upper bound and f is the 1-Lipschitz function. The K-Lipschitz function with Lipschitz constant $K = 1$, also called Metric Map, is a function that doesn't increase the distances from one metric space to the other; if f is real valued we have:

$$|f(x_1) - f(x_2)| \leq |x_1 - x_2|. \quad (3.3)$$

We can reproduce a 1-Lipschitz function with a neural network similar to the discriminator but without the sigmoid function in the last layer, where the outputs are scalar values representing how much real is the input. This new

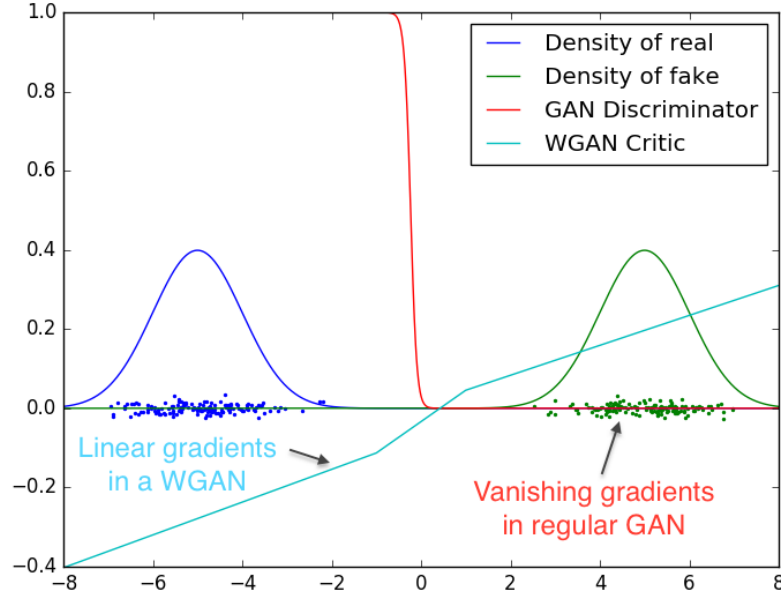


FIGURE 3.2: Graph showing the difference between vanilla GAN and Wasserstein GAN [3]. The blue line indicates the real distribution while the green one indicates the fake distribution. The red and light blue lines indicate the values of the discriminator $D(X)$ on different instances. We can see that the distance function of vanilla GAN (red line) doesn't behave very well: it changes drastically between real and fake density, generating zones of vanishing or exploding gradients where the generator is not able to learn. On the other hand, WGAN (light blue line) behaves in a better way: the gradient is smoother everywhere, helping the generator to learn correctly, even if the generator is not behaving well.

discriminator it's called critic.

Figure 3.2 shows the complete Wasserstein GAN.

The main difference with respect the Vanilla GAN is the loss function. The generator and discriminator loss become:

$$\text{critic} \quad \frac{1}{m} \sum_{i=1}^m [f(x_i) - f(G(z_i))], \quad (3.4)$$

$$\text{generator} \quad \frac{1}{m} \sum_{i=1}^m f(G(z_i)), \quad (3.5)$$

where f is a 1-Lipschitz function.

Since f must be a 1-Lipschitz function, we have to enforce the Lipschitz constraint. This reinforcement is done using a clipping to limit the discriminator's

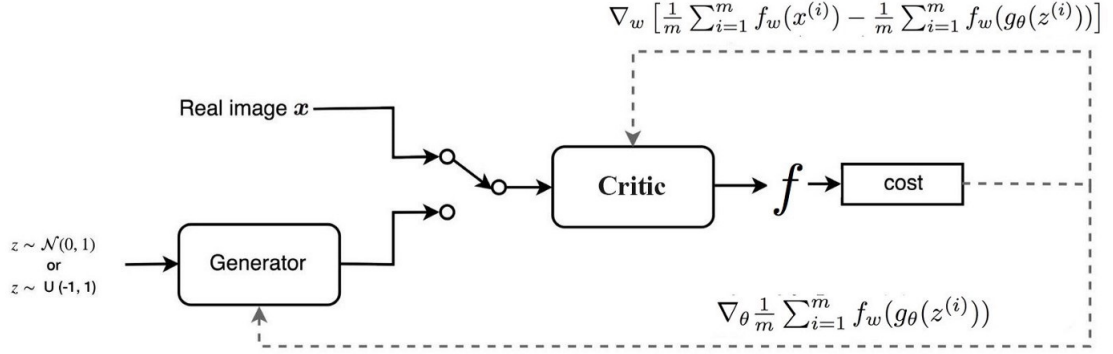


FIGURE 3.3: Schema of the Wasserstein GAN [3]. It is very similar to the vanilla GAN; the differences are the absence of sigmoid in the discriminator and the presence of the Wasserstein loss.

weights.

The Wasserstein loss encourages larger score differences between real and fake samples with respect to the min-max loss of Vanilla GAN.

Algorithm 2 shows the full Wasserstein algorithm.

Algorithm 2 WGAN, our proposed algorithm. All experiments in the paper used the default values $\alpha = 0.00005$, $c = 0.01$, $m = 64$, $n_{\text{critic}} = 5$.

Require: : α , the learning rate. c , the clipping parameter. m , the batch size. n_{critic} , the number of iterations of the critic per generator iteration.

Require: : w_0 , initial critic parameters. θ_0 , initial generator's parameters.

```

1: while  $\theta$  has not converged do
2:   for  $t = 0, \dots, n_{\text{critic}}$  do
3:     Sample  $\{x^{(i)}\}_{i=1}^m \sim r$  a batch from the real data.
4:     Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
5:      $g_w \leftarrow \nabla_w \left[ \frac{1}{m} \sum_{i=1}^m f_w(x^{(i)}) - \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)})) \right]$ 
6:      $w \leftarrow w + \alpha \cdot \text{RMSPProp}(w, g_w)$ 
7:      $w \leftarrow \text{clip}(w, -c, c)$ 
8:   end for
9:   Sample  $\{z^{(i)}\}_{i=1}^m \sim p(z)$  a batch of prior samples.
10:   $g_\theta \leftarrow -\nabla_\theta \frac{1}{m} \sum_{i=1}^m f_w(g_\theta(z^{(i)}))$ 
11:   $\theta \leftarrow \theta - \alpha \cdot \text{RMSPProp}(\theta, g_\theta)$ 
12: end while

```

In summary, the aspects that changes between the Wesserstein GAN and the Vanilla GAN are the following [6]:

- **Wasserstein loss:** Use Wasserstein loss instead the original one to train the critic and generator models.

- **Linear activation function:** Use a linear activation function in the output layer of the discriminator. Remove the sigmoid function in the last layer of the critic.
- **Clipping:** Add a clipping to force the weights of the critic model to be limited in the the range $(-0.01, 0.01)$ after the update of each batch.
- **More critic updates:** For each generator's iteration, update the discriminator 5 times.
- **RMSProp:** Use the Root Mean Square Propagation (RMSProp) (a type of gradient descent) with a learning rate of 0.00005 and zero momentum.
- **Batch size:** Use a batch size of 64.

3.2 Continual learning Methods

Continual Learning is a field of machine learning where the goal is to make the models continually learn new data. Continual learning is strongly linked with the problem of catastrophic forgetting.

Catastrophic Forgetting

Catastrophic Forgetting, also called catastrophic interference is a frequent problem of several standard models of Artificial Neural Networks. It is a subject also of connectionist literature and cognitive science. Catastrophic Forgetting can be defined as the *tendency of an artificial neural network to completely and abruptly forget previously learned information upon learning new information* [30].

Catastrophic forgetting, or more in general "interference" in connectionist literature, can also be observed in natural learning systems but in artificial neural networks is less gradual and more catastrophic. Interference, from a biological point of view, is the phenomenon of the diminishing of existing memories caused by subsequent learning.

In an apparent effort to explain the problem that traditional artificial neural networks have had with catastrophic forgetting, the stability-plasticity problem is often promoted as a dilemma: learning needs plasticity for adding new knowledge but also at the same time stability avoid the forgetting of previous knowledge. This notion has often been pushed, even in light of natural systems, which have obviously overcome the problem.

The problem of catastrophic forgetting arises from the fact that machine learning algorithms are based on the fundamental assumption of the Independent and Identically Distributed (I.I.D.). This assumption states that the training data should be independent and identically distributed; in other words training the data should be evenly random. The problem is that this assumption is not observed in a real-world environment, indeed the randomness of data can change over time: in some moments some examples are more frequent and in another moment other examples are. If we consider the distribution of the whole dataset and if the I.I.D. is not observed, the actual distribution of examples given in input to the model is a subset of the original distribution, and this subset can change over time. The absence of the I.I.D. constraint leads to a model that doesn't work properly, causing the problem of catastrophic forgetting.

In a normal condition with I.I.D., the examples of the whole dataset are continuously shuffled, letting the model adapt to all of the possible patterns of examples together and letting it find good weights. These weights are computed as the result of a good tradeoff between all the patterns. These weights will not be optimal for every single pattern but they will be a good compromise between all of them (the model doesn't specialize to one pattern but generalizes on multiple ones). However, if new examples belonging to a new pattern are given in input to the model, the I.I.D. assumption is not valid anymore and the model will start to adjust the weights to an optimal value for just that new pattern. After some time, the weights are so changed that it's like the model's weights are entirely adapted to just the new pattern; consequently, the model forgets how to deal with the former (the weights are like a memory where the model memorizes how to respond to the patterns) [9].

Image 3.4 shows an example of catastrophic forgetting.

Dealing with the catastrophic forgetting problem is not trivial. An old strategy to outflank the catastrophic forgetting problem is to completely re-train from scratch the model adding to the training set the new pattern's examples.

Continual Learning

Continual learning (CL), also called lifelong learning, is a machine learning field where the goal is to improve traditional machine learning algorithms adding the capability of continuously learning new data [26]. Continual learning addresses the catastrophic forgetting problem and the Stability-Plasticity Problem and tries to solve them.

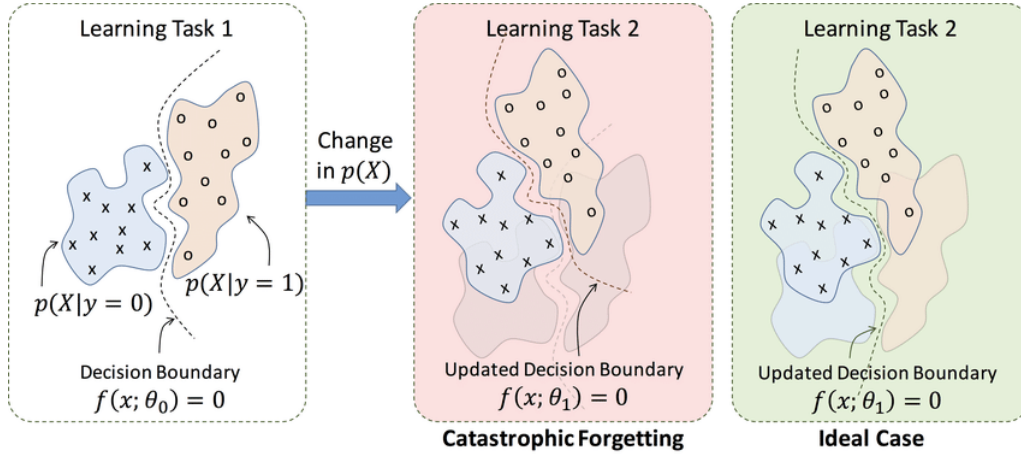


FIGURE 3.4: Illustration of catastrophic forgetting. If we consider showing to a model a multitude of different tasks in sequence, each of them corresponding to a different distribution, we will be in a situation of absence of the I.I.D. condition. In this case, the model can suffer from catastrophic forgetting. In the image, there is a binary classification model that has to separate samples into two different areas. If catastrophic forgetting occurs, the model will forget the old decision boundary and therefore the classification of the old examples.

The basic concept of continual learning is to consider a training set composed by several tasks, each of them corresponding to a dataset with different distribution with respect to the others. The model is trained sequentially on all the tasks and should learn all of them. However, in a condition where there are multiple tasks, the I.I.D. assumption on data is not valid, and this leads to the problem of catastrophic forgetting, where, during training, after some time, the model will remember only the latest tasks seen, forgetting all the previous ones. Therefore, the difficult part of CL is to develop approaches that avoid the problem of catastrophic forgetting.

In an environment where there are multiple tasks in sequence to learn there are 3 main performance measures (Lopez-Paz and Ranzato [27]):

- **Average Accuracy:** It measures the accuracy of the model in a specific task.
- **Forward Transfer:** It measures how much learning a task t influences the performance of another future task $t + k$. If FT is positive it means that performance on task k increments if task t is previously learned.

- **Backward Transfer:** It measures how much learning a task t influences the performance of another past task $t - k$. If BT is positive it means that performance on task k increments if task t is learned.

Human intelligence can be defined as:

Goal-directed adaptive behavior
— Sternberg [42]

So the core concept is that human beings are able to cognitively adapt to new circumstances. Adaptation is very important and some traditional machine learning algorithms are already able to adapt to new data, but there is also another aspect of human intelligence linked with adaptation to take into account, that is scalability. The scalability is the capability of a cognitive system to scale as more and more data is accumulated maintaining the memory sustainable [26]. Indeed, the human brain is able to adapt to new data without forgetting the older one. This is not always true because people forget a lot of things, especially when getting older because of brain deterioration, but even old people with memory leaks are able to remember very old information or experiences, like childhood ones. This long-term memory is absent in traditional machine learning algorithms and continual learning can solve this problem.

There is also to consider that human memory is quite complex since it is influenced by emotions: when there is a very strong emotional event (good or bad) the brain gets more active and record new information with more intensity. However, this is another subject.

An important paper that summarizes continual learning approaches is Van de Ven and Tolia [43]. This paper considers two kinds of classification about continual learning literature. The first classification is the type of categorization scheme:

- **Multi-headed layout:** In this layout, each task has an identity that must be known. In this case, we have a separated output layer for each task.
- **Single-headed layout:** In this layout, tasks don't have an identity. In this case, we have a single output layer for all the tasks.

The second classification is the kind of scenario:

- **Task-incremental Learning:** In this scenario, the model is informed about which task it is currently working on. A typical model for this scenario has a multi-headed layout with a separated output layer for each task since each task has an identity. This is the easiest scenario.

- **Domain-incremental Learning:** In this scenario, the model is not informed about the current task upon which it is training. However, the only model's goal is to solve the task without having to guess which task it is.
- **Class-incremental Learning:** This scenario is similar to the domain-incremental learning with the difference that the model has also to guess the task upon which it is trained.

In Continual Learning literature there are 3 main methods:

- **Regularization:** It consists in using a specific regularization applied to the loss of the model in order to penalize the weights mutation and so to consolidate previous knowledge. This regularization can be differentiated based on the currently running task. For instance, regularization approaches are Elastic Weight Consolidation (EWC) from Kirkpatrick et al. [21], Learning Without Forgetting (LwF) from Li and Hoiem [24], Large Scale Incremental Learning from Wu et al. [46].
- **Architectural:** It consists in training a different portion (sub-network) of the model for each task. In this way, there is not the problem of overwriting all the network weights that leads to the problem of catastrophic forgetting. There are several strategies about selecting the sub-network: choosing the neurons randomly for each task (Masse, Grant, and Freedman [28]), choosing the neuron based on learning algorithms using gradient descent (Serra et al. [39]) or evolutionary algorithms (Fernando et al. [11]). This approach works only with Task-incremental learning.
- **Rehearsal/Replay:** It consists in saving some sort of data (pseudo-data) representing the data of the previous tasks and then extending the training data of the next tasks with it. It's like proposing again a small amount of older tasks when new tasks arrive. The simplest kind of replay is the "exact replay", where plain data of old tasks is memorized and replayed. For instance, rehearsal approaches are Experience Replay from Ratcliff [34], Dark Experience Replay from Buzzega et al. [8], Gradient Episodic Memory (GEM) from Lopez-Paz and Ranzato [27], Deep Generative Replay from Shin et al. [40], Incremental Classifier and Representation Learning (iCaRL) from Rebuffi et al. [35].

We will see two continual learning methods in detail: Elastic Weight Consolidation and Experience Replay.

Elastic Weight Consolidation

Elastic Weight Consolidation (EWC) [21] is a quite important Continual Learning method in the last few years.

Elastic Weight Consolidation's basic concept to avoid catastrophic forgetting is to make the learning slower for the weights that were important in the previous tasks. So, this method computes the magnitude of each parameter and penalizes further changes of them if they were important in the past. This concept is also similar to Synaptic Intelligence [49].

This concept is inspired by human intelligence. Indeed, in the brain synapses are not static but changes over time: they can get stronger or weaker based on our experiences. There is also a phenomenon called synaptic consolidation where the plasticity of synapses is reduced in order to not modify them. This happens if these synapses are very important considering in particular their current strength (value), this is why they shouldn't change. There are also studies that say that synapses store, aside from the weight, also the uncertainty about that weight. If the uncertainty is high, the plasticity of the synapse increases.

If we consider two different tasks, the goal is to find parameters corresponding to a good tradeoff of performances between the two tasks. This is possible considering that there is evidence that among all the possible combinations of parameters, many of them would lead to very similar performances. Therefore, it's easier to believe there could be some parameter configurations that are pretty good for both tasks, without giving up on the best individual performances. This is true in particular with networks with a redundant amount of nodes. So if there are multiple tasks to learn in sequence, EWC would force the current parameters to be centered around the best parameters of the previous tasks. This constraint corresponds to a quadratic penalty and the intensity of this penalty would change for each different weight based on its importance. This penalty can be seen as a spring binding the parameters, and this leads to the name "elastic".

Image 3.5 shows how EWC behaves with two tasks.

If we consider the training a NN from a probabilistic point of view, the optimization of parameters would correspond to the conditional probability $p(\theta|\mathbb{D})$. The log of this quantity can be computed with the Bayes' rule:

$$\log p(\theta|\mathbb{D}) = \log p(\mathbb{D}|\theta) + \log p(\theta) - \log p(\mathbb{D}), \quad (3.6)$$

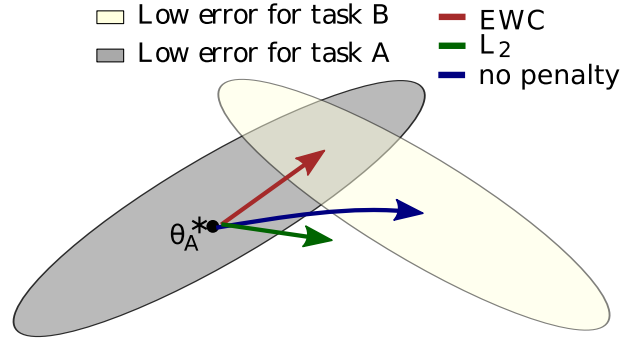


FIGURE 3.5: Illustration of how EWC behaves with two tasks A and B. The colored areas indicate parameter regions where the model has good performances; in particular the grey for task A and white for task B. Initially, the model's parameters correspond to θ_A^* and, after a normal training on task B without penalties, the parameters would change a lot moving away from the A region and going inside the B region. This is not good since it means that the model completely forgot how to behave correctly with the previous task A, showing signs of catastrophic forgetting. If we use L_2 and constrain each weight with the same coefficient, the penalization brings the penalties outside both the regions, and this is not good either. Finally, if we use EWC, we can see that the parameters would move in a shared region between A and B. This means that the model is able to partially learn B (it is not the best possible position) and, at the same time, not completely forget A.

where $p(\theta)$ is the prior probability of the parameters and $p(\mathbb{D}|\theta)$ is the likelihood of the data given the parameters. The log of the former quantity $\log p(\mathbb{D}|\theta)$ is the negative of the loss function $-\mathcal{L}(\theta)$. If we assume to split the data into two tasks $\mathbb{D}_A$ and $\mathbb{D}_B$, rearranging previous equation we obtain:

$$\log p(\theta|\mathbb{D}) = \log p(\mathbb{D}_B|\theta) + \log p(\theta|\mathbb{D}_A) - \log p(\mathbb{D}_B). \quad (3.7)$$

Inside $p(\theta|\mathbb{D}_A)$ there is the information of the importance of the parameters on previous task A. To make the posterior probability $\log p(\theta|\mathbb{D})$ tractable it is approximated to a Gaussian distribution with θ_A parameters as mean and the Fisher information matrix F as diagonal precision. The function that we want to minimize is:

$$\mathcal{L}(\theta) = \mathcal{L}_B(\theta) + \mathcal{L}_A^{EWC}(\theta), \quad \text{where} \quad \mathcal{L}_A^{EWC}(\theta) = \frac{\lambda}{2} \sum_i F_i (\theta_i - \theta_{A,i}^*)^2, \quad (3.8)$$

where $\mathcal{L}_B(\theta)$ is the loss of B, λ is a scale factor that manipulates the significance of the old task with respect to the new one, i is the index of the parameters. In

this loss expression are just considered two tasks A and B, but we can extend the expression with a multitude of tasks. For each of the $K - 1$ previous tasks, a new $L_k^{EWC}(\theta)$ corresponding EWC term is added to the loss function.

The Fisher information matrix can be computed by taking the diagonal elements of the True Fisher Information (Ven and Tolias [44]). The diagonal values can be computed in the following way:

$$F_{i,i} = \frac{1}{|\mathbb{D}_A|} \sum_{\mathbf{x} \in \mathbb{D}_A} \left(\frac{\partial \log p(Y = y|\mathbf{x}; \theta_A^*)}{\partial \theta_i} \right)^2. \quad (3.9)$$

The estimation of the Fisher Information matrix is computed once for each task and the matrix size depends on the number of examples of the training data of the corresponding task.

The Fischer information matrix properties are:

- It is the same as the losses double derivative near a minimum.
- It is computable from first-order derivatives thus computationally light.
- It is positive semi-definite.

In the end, in EWC, three values are stored for each synapse (the weight, its variance, and its mean) and two quantities for each task (the best parameters, the Fisher Information Matrix).

Online EWC

If we recall the expression of the EWC loss function that we have previously seen 3.8 and we extend it summing a multitude of EWC losses for each previous $K - 1$ task, EWC doesn't scale well. Indeed, each EWC loss term requires saving an historical reference term θ_k^* (the best parameters for task k) and a Fischer information matrix F_k and, if we are dealing with a bigger model with a lot of tasks, it becomes memory consuming.

A variation of EWC to solve this problem of optimization is proposed by Schwarz et al. [38], that is called *Online EWC*. The main concept to solve this memory problem is to just consider the optimal parameters after the most recent task and to sum up the previous tasks' Fisher Information matrices, so we

obtain the following expression:

$$\mathcal{L}(\theta) = \mathcal{L}_{k+1}(\theta) + \mathcal{L}^{EWC}(\theta), \quad \text{with} \quad \mathcal{L}^{EWC}(\theta) = \frac{\lambda}{2} \sum_i S_{k,i} (\theta_i - \bar{\theta}_{k,i}^*)^2, \quad (3.10)$$

$$\text{where} \quad \bar{\theta}_{k,i}^* = \frac{P_{k,i}}{S_{k,i}}, \quad S_{k,i} = \sum_{k=1}^k F_{k,i}, \quad P_{k,i} = \sum_{k=1}^k F_{k,i} \theta_{k,i}^*, \quad (3.11)$$

where $\theta_{k,i}^*$ is basically the best value of the parameter i after finishing training on task k .

There is also to consider that the initial iterations of the discriminator will probably be not optimal and this will cause EWC to possibly lock the discriminator forever to a bad initialization. There is also to consider that the unlimited addition of $\mathcal{L}^{EWC}(\theta)$ terms will cause a limitless grow of the EWC regularization term, and this could prevent further changes of the parameters.

To build a forgetting mechanism that discard the poor initialization of the discriminator and keep the EWC regularization term bounded, a *discount factor* γ is considered. The S_k and P_k terms are updated every α step in an online way:

$$S_{k,i} = \gamma S_{k-1,i} + F_{k,i}, \quad P_{k,i} = \gamma P_{k-1,i} + F_{k,i} \theta_{k,i}^*. \quad (3.12)$$

Thanks to these characteristics, Online EWC is able to memorize just the last parameters and only one Fisher Information Matrix by accumulating them over tasks (the quantities saved are just a running average of S_k and P_k).

Experience Replay

If we consider the classic continual learning scenario, where the model has to learn multiple tasks in sequence, the distribution is non-stationary and the data doesn't follow the I.I.D. assumption. The basic concept of **Experience replay** (ER) to solve this problem is to give in input to the model a stationary distribution of all the tasks seen so far, and thus consider a training dataset that follows the I.I.D. assumption. To obtain this distribution, Experience Replay works in the following way: the examples of past tasks are stored in a memory M and these examples are added to the current batch of training. Therefore, past and new examples are mixed together obtaining a stationary distribution that approximates the whole dataset D . If we consider the optimization objective of a classic offline supervised learning over the dataset D and we substitute it with

M, we obtain:

$$\theta = \arg \min_{\theta} \mathbb{E}_{(x,y) \sim D} [L(x, y)] \quad (3.13)$$

$$\approx \arg \min_{\theta} \mathbb{E}_{(x,y) \sim M} [L(x, y)], \quad (3.14)$$

where x, y are the pairs of examples, L is the loss function. There is to consider that the memory M is able to represent with a good approximation the whole distribution D only if the examples stored capture all the variation of the past tasks. Since examples are stored randomly, this condition can't be guaranteed.

The algorithm 3 shows how the Experience Algorithm works. Each iteration, the current training example x, y are added $k-1$ examples from the memory, obtaining a batch B of k size. So, in the buffer, there is 1 new example and $k-1$ old examples. Then, the parameters are computed with stochastic gradient descent over batch B . Finally, the memory is updated.

Algorithm 3 Experience Replay (ER) with Reservoir Sampling

```

procedure TRAIN( $D, \theta, \alpha, k$ )
   $M \leftarrow \{\}$ 
  for  $t = 1, \dots, T$  do
    for  $(x, y)$  in  $D_t$  do
      // Draw batch from buffer:
       $B \leftarrow \text{sample}(x, y, k, M)$ 
      // Update parameters with mini-batch SGD:
       $\theta \leftarrow \text{SGD}(B, \theta, \alpha)$ 
      // Memory update:
       $M \leftarrow M \cup \{(x, y)\}$  (algorithm 4)
    end for
  end for
  return  $\theta, M$ 
end procedure

```

The buffer M is updated with **reservoir sampling**, as we can see in algorithm 4. M indicates the total memory size. Initially, it is empty and thus, if M is greater than the total examples seen till now N , the memory is filled up. Otherwise, if $M < N$ it means that the memory is full and therefore, each iteration, a random example in memory is substituted with the current one. The memory index j is randomly computed between 0 and N but, since N increases endlessly, the probability of finding an acceptable index (M/N) decreases (the

index is acceptable when $j < M$). This means that, as training time passes and more and more examples are shown, the examples in the memory buffer are substituted less and less and after some time they become basically permanent.

Algorithm 4 Reservoir Sampling with Algorithm R

```

procedure RESERVOIR( $M, N, x, y$ )
  if  $M > N$  then
     $M[N] \leftarrow (x, y)$ 
  else
     $j = \text{randomInteger}(\text{min} = 0, \text{max} = N)$ 

    if  $j < M$  then
       $M[j] \leftarrow (x, y)$ 
    end if
  end if
  return  $M$ 
end procedure

```

The description of Experience Replay is taken from Riemer et al. [36].

Other continual learning methods

Learning without Forgetting (LwF) (Li and Hoiem [24]) [18] consists in using as pseudo-data an input-target where the input comes from the current task while the label is computed using the model trained on old tasks. The label consists of the predicted probabilities of all the classes. This pseudo-data is replayed on the model and the goal is to match the predicted probabilities with the labeled ones. This method was previously used to distill information from a large model to a smaller one.

Incremental Classifier and Representation Learning (iCaRL) (Rebuffi et al. [35]) is an approach that uses an ANN for feature extraction performing on it a classification with nearest-class-mean rule. Class means are calculated from the stored data. In order to prevent catastrophic forgetting on the feature extractor network, it replays the stored data with a distillation of the current one during feature extractor training.

Deep Generative Replay (DGR) [40] consists in generating new samples with a generative model trained on all the tasks in sequence. Then, they are

paired with the label of the most probable class, computed with the main classification model. A variation is to label the samples with the probabilities of all the classes, obtaining distillation.

4 Case study

The main focus of this work is based on the work of Liang et al. [25]. This paper addresses the problem of Catastrophic Forgetting in Generative Adversarial Networks and tries to solve it using Continual Learning methods. Moreover, the paper considers also the dataset generated by GANs as a good testing ground for Continual Learning algorithms.

In particular, the authors use two of the most popular methods: Elastic Weight Consolidation and Intelligent Synapses. In this paper 4 experiments are carried on: the first one consists of a DCGAN trained on the MNIST dataset, the second consists in training a vanilla GAN on an eight Gaussian distribution, the third TTUR is trained on CelebA and CIFAR-10, the fourth is a text generation of COCO captions using textGAN. This work will focus on the eight Gaussian distribution experiments.

4.1 Mode Collapse and Catastrophic Forgetting in an eight Gaussians distribution

The eight Gaussian distribution is a mixture of Gaussians that is taken from the Metz et al. [31] setup. It consists of data that is evenly distributed among eight 2-dimensional Gaussian distributions, each with a covariance of $0.02I$ and arranged in a circular shape of radius 2. The eight Gaussians are collocated in the same position of vertices of an octagon. Image 4.1 shows the distribution.

This eight Gaussian distribution constitutes the real data that the generator has to simulate in order to fool the discriminator. If we try to train a vanilla GAN model on this particular dataset, it doesn't work very well. Indeed, the generator's distribution, after some adjustments in the initial iterations, concentrates in just one of the eight Gaussians and, as time passes, the generator's distribution changes its position, centering in another one of the eight Gaussians.

Since GANs consist of a zero-sum game where an unbalanced condition between the generator and discriminator is very probable, this can cause an



FIGURE 4.1: Illustration of the eight Gaussian distribution, with radius 2 and covariance $0.02I$. It is composed by several red dots obtained by randomly sampling from the distribution (each dot is a sample).

endless cat-and-mouse game where the convergence is not reached: initially, the generator overfits in one of the eight Gaussians to fool the discriminator and the discriminator follows it in order to discriminate it, overfitting in that Gaussian too; after that, the generator changes its position to fool the discriminator again and the cycle restart.

This is the Mode Collapse phenomenon; indeed, the generator learns to map the entire latent space into a subset of the real distribution, failing to correctly represent the wholeness of the real distribution. In this case, the subset is just one of the eight Gaussian.

This problem of mode collapse is caused by the fact that GANs are predisposed on their own to that problem (zero-sum game) and the fact that the eight Gaussians distribution is a multi-modal data where the generator struggles to avoid learning just one mode.

The first row of image 4.2 shows the Mode Collapse phenomenon. In the end, the final effect of Mode Collapse is that the generator's distribution changes over time moving around the Gaussians in a circular path.

However, there is an interesting detail to notice. As we said, the generator changes its distribution on one of the eight Gaussians at a time, starting with the first one followed by the second, and so on. When it reaches the eighth Gaussian, the Generator oddly changes its distribution again in the first Gaussian, and the cycle restarts. This is strange since the discriminator, the first time the generator moved on the first Gaussian, should have learned to kick it out from that Gaussian since it *was previously able* to do it. But, the second time the generator passes on the same Gaussian, the discriminator is not able to kick it immediately out, arresting this endless cat-and-mouse game. It seems that the discriminator is wasting time trying to learn again from scratch that

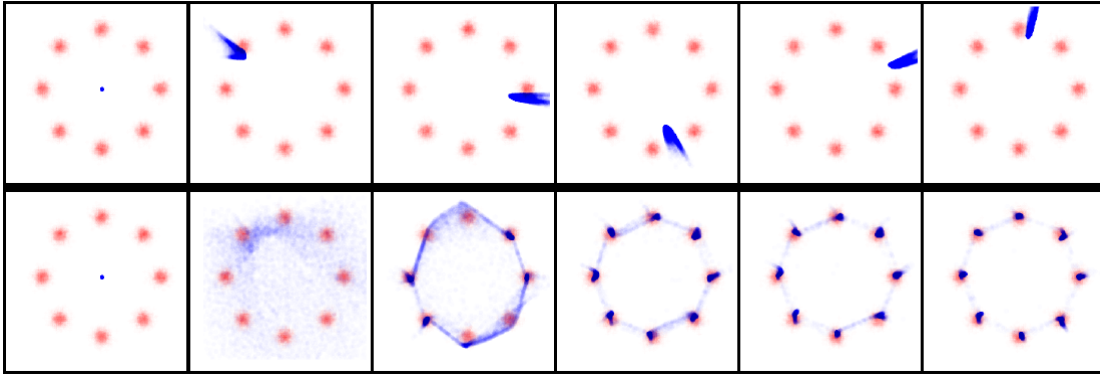


FIGURE 4.2: Illustration of two possible evolutions of the GAN training. In the first row of the grid we have the case of a vanilla GAN while, in the second row, we have an enhanced GAN that works correctly. They must be read from left to right and each image corresponds to a timestamp of the training along time. The red zones represent the real eight Gaussians distribution while the blue ones the generator's distribution. In the case of vanilla GAN, as we can see, there is the problem of Mode Collapse as the generator focuses on one of the Gaussians (or near it), changing its position multiple times. In the case of an enhanced GAN, the generator's distribution spreads correctly around all the Gaussians, that is our objective.

generator's modal (the overfitting on that Gaussian).

As Liang et al. [25] paper says, it seems that the discriminator suffers from catastrophic forgetting. This makes sense since the Mode Collapse phenomenon resembles the classic Continual Learning scenario where the model has to learn sequentially a multitude of tasks. In the Mode Collapse case, each mode is a different distribution (a subset of the original one) that can be interpreted as a different task; in general, it's like there is a different task each time the generator changes its distribution. In this scenario of multiple tasks, the data doesn't follow the I.I.D. assumption and, as we have previously seen, it causes the problem of Catastrophic Forgetting.

The discriminator has to learn to discriminate the generator in all the eight Gaussians but, after it learned some of them, it starts to forget the older ones and the generator can easily fool him by continuously focusing on old Gaussians that the discriminator forgot. Because of this, the model never converges and never reaches a condition where the generator learns to distribute over all the eight Gaussians together, like in image 4.2 in the second row.

Based on these findings, Liang et al. [25] suggests that a way to solve Model Collapse is to apply Continual Learning methods on the discriminator. In this way, the generator is not able to fool the discriminator again by leveraging on

its forgetfulness and it is forced to change strategy, hopefully distributing and learning all the Gaussians together.

4.2 Implementation

In this section we will go into detail about the implementation of the various algorithms, starting from the Vanilla GAN. Then, in this work, to avoid catastrophic forgetting and therefore to avoid Mode Collapse Elastic Weight Consolidation and Experience Replay are implemented. In order to make a comparison and see how well the previous two continual learning methods works, also L2 regularization and Wasserstein GAN are implemented.

Vanilla GAN considerations

Vanilla GAN is the basic GAN implemented by Goodfellow et al. [15]; the code is taken from [here](#) and it is a PyTorch version of the original code of Metz et al. [31].

Network architecture and experimental details for the experiment in Section 3.1 are as follows:

The main characteristics of the model are the following:

- The dataset is a mixture of 8 Gaussians with a standard deviation of 0.02, disposed around a circle of radius 2.
- The generator's network is a fully connected network composed of 2 hidden layers with size 128 followed by ReLu activation function and then a linear projection to 2 dimensions. The initialization of the weights set them to be orthogonal with a scaling factor of 0.8.
- The discriminator is a fully connected network where the first step consists in scaling by a factor of 4 followed by one layer of size 128 with ReLu and finally a linear layer with output size 1 that acts as a Boolean.
- The generator minimizes:

$$\mathcal{L}_G = \log(D(x)) + \log(1 - D(G(z))), \quad (4.1)$$

while the discriminator minimizes:

$$\mathcal{L}_D = -\log(D(x)) - \log(1 - D(G(z))), \quad (4.2)$$

where x is a sample of the real data while z is a sample of the latent space with normal distribution: $z \sim \mathcal{N}(0, I_{256})$.

- The generator and discriminator are optimized using Adam [20] with learning rate $1e-4$ and $\beta_1 = 0.5$.
- The training is done by alternating the generator and the discriminator.
- The criterion used is the BCEloss.

The parameters of both the discriminator and generator are memorized for each different training.

L2 loss

L2 regularization is implemented on the parameters update algorithm, using Adam with a weight decay of 0.01. Equation 2.25 shows the expression of L2 contribution on the stochastic gradient descent.

Wasserstein GAN considerations

The Wasserstein GAN is implemented following the Arjovsky, Chintala, and Bottou [3] paper and, in particular, taking into account the following aspects:

- The standard min-max loss of the vanilla GAN is substituted with the Wasserstein one. The implementation is taken [here](#). The discriminator has to minimize the $-D(x) + D(G(z))$ while the generator has to minimize $-D(G(z))$. A mean of the discriminator outcomes is computed (the number of outcomes depends on the batch size).
- The sigmoid function in the last layer of the discriminator is removed in order to have a linear activation function in the output layer.
- A clipping between (-0.01,0.01) is applied on the weights of the critic model each discriminator's update.
- Increase the discriminator 5 times with respect to the generator one.
- Apply the Root Mean Square Propagation instead of the Adam Optimization Algorithm [20].
- The batch size is 64.

EWC considerations

Elastic Weight Consolidation is one of the most popular Continual Learning algorithms and it can be used to solve the Catastrophic Forgetting problem and therefore also the Mode Collapse.

In particular, the Online EWC version is used and the code is taken from van2019three GitHub code.

There is an aspect to consider to implement EWC on a GAN model. EWC is a method that was originally proposed for the classic Continual Learning scenario where the model has to learn a multitude of well-defined tasks in sequence. There is however to consider that, in general, in a GAN environment there is not this scenario and it is not possible to precisely define a task; it is also not clear how long a task should be. A simple solution is to compute the total number of tasks K by dividing the number of iterations T by a constant α obtaining $K = T/\alpha$.

At the end, in the Online EWC algorithm applied on GANs there are 4 parameters to consider:

- **Alpha (α):** as we said, it indicates the size of tasks in terms of the number of iterations of generator and discriminator.
- **Lambda (λ):** it is a scale factor that indicates the importance of the old tasks with respect to the new ones. If it is high, the old tasks can strongly influence the loss.
- **Gamma (γ):** it is a discount factor that is used to control the discriminator's forgetting in the initial steps of training since it can be counterproductive.
- **Fisher Information Matrix size:** It indicates the size of the Fisher Information Matrix. Since in GANs there is not a scenario with distinct tasks, we can arbitrarily choose the size of the Fisher matrix. Higher is the value, the more accurate the Fisher Information Matrix estimation will be; however, if it is too high, the computation costs become remarkable.

Both the parameters α and γ control the time distance the discriminator remembers the old distributions of the generator, and λ determines how strong is the contribution of the memory in the discriminator's loss.

The new GAN objective with the discriminator augmented with the Online EWC loss becomes:

$$\min_{\phi} \max_{\theta} \mathcal{L}(\theta, \phi) = \mathcal{L}^{GAN}(\theta, \phi) - \mathcal{L}^{EWC}(\theta), \quad (4.3)$$

where $\mathcal{L}^{GAN}$ is the loss of the GAN, that takes as input the parameters of the discriminator θ and the parameters of the generator ϕ , and the EWC loss $\mathcal{L}^{EWC}$.

We can see the complete GAN-EWC algorithm here [5](#).

Algorithm 5 Continual learning GAN with EWC

```

1: Input: Training data  $\mathcal{D}^{\text{real}}$ , latent distribution  $p(z)$ , hyperparameters of
   continual learning  $\alpha, \gamma, \lambda$ , step size  $\epsilon$ 
2: Output:  $\theta, \phi$ , and generated samples  $\mathcal{D}^{\text{gen}} = \{x_j\}_{j=1}^N$ 
3: for  $t = 1, \dots, T$  do
4:   Noise sample:  $\{z_j\}_{j=1}^m \sim p(z)$ 
5:   Data sample:  $\{x_j\}_{j=1}^m \sim \mathcal{D}^{\text{real}}$ 
6:   % Calculate current discriminator loss
7:    $\mathcal{L}_{\theta} = \frac{1}{m} \sum_{j=1}^m [\log D_{\theta}(x_j)] + \frac{1}{m} \sum_{j=1}^m [\log(1 - D_{\theta}(G_{\phi}(z_j)))]$ 
8:   % Update history buffer for previous tasks every  $\alpha$  steps
9:   if  $\text{mod}(t, \alpha) = 0$  then
10:    for parameters  $\theta_i$  in  $\theta$ :
11:       $Q_i = \left(\frac{\partial \mathcal{L}_{\theta}}{\partial \theta_i}\right)^2$  %  $Q$  is the Fisher for EWC
12:       $S_i = \gamma S_i + Q_i$ 
13:       $P_i = \gamma P_i + Q_i \theta_i^*$ 
14:       $\bar{\theta}_i^* = \frac{P_i}{S_i}$ 
15:    end if
16:    % Update discriminator parameter, adding EWC
17:     $\tilde{\mathcal{L}}_{\theta} = \mathcal{L}_{\theta} - \frac{\lambda}{2} \sum_i S_{k,i} (\theta_i - \bar{\theta}_{k,i}^*)$ 
18:     $\theta_{t+1} \leftarrow \theta_t + \epsilon_t \frac{\partial}{\partial \theta_i}$ 
19:    % Update generator parameter
20:     $\mathcal{L}_{\phi} = \frac{1}{m} \sum_{i=1}^m [\log(1 - D_{\theta}(G_{\phi}(z_i)))]$ 
21:     $\phi_{t+1} \leftarrow \phi_t - \epsilon_t \frac{\partial}{\partial \phi_i}$ 
22: end for

```

Experience Replay considerations

Experience Replay is implemented following the algorithms 3 and 4 of Riemer et al. [36] paper. To summarize, there are three main phases: a batch is sampled from the buffer (it is mixed with the batch of current examples and then it is given in input to the discriminator), the discriminator's parameters are updated with stochastic gradient descent, the memory buffer is updated with reservoir sampling.

In Experience Replay, we just save the generator's examples, since the discriminator has to remember in particular them.

There are two important implementation details to consider:

- During the batch sampling from the buffer, this batch of old examples is mixed up with a batch of new examples with a size greater than one. (in the algorithm 3 there is just one sample).
- During the ER buffer update, the reservoir sampling algorithm is applied N times where N is the batch size. Therefore, each iteration, the memory is updated with the whole batch of fake samples.

The main parameters of Experience Replay are:

- **Memory Size:** the size of the ER batch where the fake examples are stored. Higher is the memory, more examples the discriminator is able to "remember".
- **Batch size:** the batch size of the old examples sampled from memory and the batch size of the new examples. These two quantities can be calibrated to find a good tradeoff between the influence of past examples and new examples.
- **Aging start:** the buffer activates and starts to memorize after a specific number of training iterations. Before the starting iteration, the batch of old examples is empty while the batch of new examples is unchanged. Higher is the starting iteration, later the Experience Replay contribution joins the equation, during training.

Together with Experience replay we also tried to add a gradient clipping of the discriminator's parameters. This is done to get a higher stabilization of the discriminator.

The buffer is memorized for each different training.

5 Empirical Study

In this chapter, we will focus on the results of the experiments done with the GAN model using the various methods to better stabilize it, such as L2 regularization and WGAN, and methods of catastrophic forgetting, such as Elastic Weight Consolidation and Experience Replay. The experiments are in particular three:

- A comparison between the above approaches 5.2.
- A comparison between combinations of the above approaches 5.3.
- Observing the Experience Replay’s behavior by varying its buffer size 5.4.

The following playlist shows the evolution of the training of the different approaches of table 5.2: [playlist](#)¹. Figure 5.1 shows a frame of the GAN-ER video.

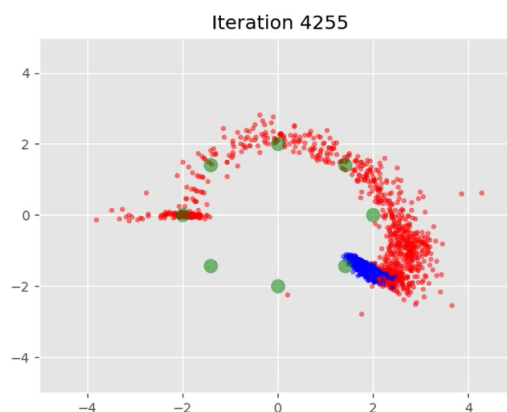


FIGURE 5.1: [Playlist](#) videos description: the green dots indicate the eight Gaussians, the blue dots indicate the generator’s distribution, the red dots indicate the memorized samples in the ER buffer.

The training is done with 25000 iterations, following Liang et al. [25] experiments, since they are enough to capture all the evolution of the training;

¹https://youtube.com/playlist?list=PL9kKvDeCWmjKKEmSX00mqcnXekKx_iGuD

indeed, usually, past that iteration, the generator's distribution doesn't change significantly. We repeat the evaluation 20 times since, more samples we have, the better the evaluation of the model will be. This is because every single training behaves partially in a casual way: some of them are lucky, conducting to a model that behaves correctly; others are not, conducting to a model that doesn't behave well. During the evaluation, only the 10 best trainings were considered to discard the unluckiest cases.

5.1 Evaluation methods

Several estimators can be used to compute the similarity between two distributions:

- KL-divergence computed with the k-Nearest-Neighbour.
- KL-divergence computed on the (one with log and the other without) probabilities of the bi-dimensional histograms of the two distributions (the probability is computed by dividing each element of the histogram with the total sum of them).
- Inception Score (ICP) that computes a rough diversity of the samples' quality.

Symmetric KL-divergence

To estimate the performance of the various models we computed the symmetric Kullback–Leibler divergence between the real eight Gaussians distribution and the fake generator's distribution.

In statistics, the **Kullback–Leibler divergence** (KL-divergence) D_{kl} is a measure of the difference between two probabilities distributions. The two distributions are usually called P and Q , where P represents an observation or measurement while Q represents a theoretical model; Q can be mentioned as the reference probability distribution. KL-divergence can also be interpreted as how much information loss there is by approximating distribution P with Q . If the KL-divergence of two distributions is 0, it means that they have the same quantity of information.

To measure the quantity of information that there is in data we use the *entropy* H :

$$H = - \sum_{x \in \mathcal{X}} P(x) \log(P(x)), \quad (5.1)$$

where x are samples of the probability space $\mathcal{X}$ and P is a discrete probability distribution defined on $\mathcal{X}$. If we use the log base 2 in the entropy equation, the entropy can be interpreted as the minimum quantity of bits needed to encode samples of P with a code optimized for P .

Now that we know how to quantify the information, we can compute a difference between the information of two distributions. KL-divergence is a modification of the entropy formula, adding a new probability distribution Q , obtaining the following:

$$D_{KL}(P||Q) = \sum_{x \in \mathcal{X}} P(x)(\log(P(x)) - \log(Q(x))) = \sum_{x \in \mathcal{X}} P(x) \log \left(\frac{P(x)}{Q(x)} \right). \quad (5.2)$$

So, at the end, we can say that, by using a logarithm base 2, the KL-divergence can be interpreted as the average difference between two quantities: the first one is the quantity of bits needed to encode the samples of P through a code optimized for Q ; the second one is the quantity of bits needed to encode samples of P through a code optimised for P instead [33]. KL-divergence can also be computed as the expectation, taken using P , of the difference of the logarithms of P and Q .

There is to consider that the KL-divergence is asymmetric, therefore $D_{KL}(P||Q) \neq D_{KL}(Q||P)$. To avoid this unpleasant characteristic, we can consider the **symmetric KL-divergence** version, defined by Kullback and Leibler themselves, that is the following:

$$D_{symKL}(P, Q) = D_{KL}(P||Q) + D_{KL}(Q||P). \quad (5.3)$$

The KL-Divergence estimator can be implemented using the K-nearest-neighbours probability density estimator, following Wang, Kulkarni, and Verdú [45] paper.

In this work, the evaluations are done using the symmetric KL-divergence implemented with K-Nearest-Neighbour where parameter $k = 5$ using a batch of one million samples.

The Symmetric KL-divergence is computed for each of the 10 trainings of the model, and then the **Mean** and **Standard Deviation** (SD) of the scores of the trainings is computed. Lower is the mean, better is the generator in terms of how similar is its distribution with respect to the eight Gaussians distribution. Higher is the standard deviation, more variability there is between trainings. The ideal model has a value of zero in both mean and standard

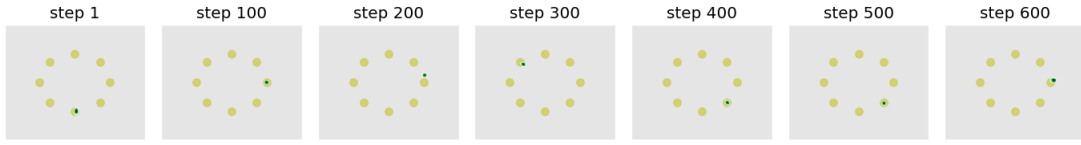


FIGURE 5.2: Illustration of some iterations during the training of a Vanilla GAN on the eight Gaussians distribution. Each image shows the eight Gaussians distribution in yellow and the generator's distribution in green over an interval of 600 iterations with 100 iterations of steps. As we can see, the model suffers from Mode Collapse. This interval of iteration is taken in the middle of the training and not at the beginning (in the first iteration, the generator starts in the middle of the distribution following by a brief period of rearrangement that then leads to this Mode Collapse behavior).

deviation.

5.2 Comparison between approaches

In this section we will focus on the performances of how the single models behave on the eight Gaussians distribution; the models tested are Vanilla GAN 5.3, GAN with L2 regularization 5.4, Wasserstein GAN 5.5, GAN with Elastic Weight Consolidation 5.6 and finally GAN with Experience Replay 5.7.

The goal is to understand if Continual Learning methods can improve the Vanilla GAN: if there is an improvement it means that the hypothetical correlation between Node Collapse and Catastrophic Forgetting can be true, otherwise not. Moreover, we will compare the performances of CL methods (EWC/ER) with the baseline (L2/WGAN) to see which one behaves better.

The chosen ER hyperparameters for GAN with Experience Replay (5.7) are a batch size of new samples of 384, a batch size of buffer samples of 128, a buffer size of 5000, 1000 as starting iteration of memorization, 0.01 of gradient clipping.

Each illustration shows the 10 best trainings of that model, where the green dots are the 8 Gaussians of the real distributions while the several blue dots are samples from the generator's distribution.

Table 5.1 shows the mean and standard deviations of the symmetric KL-divergence score of the models.

Symmetric KL-divergence	
Model	Mean $\pm$ SD
Vanilla GAN	16.43 ± 6.58
GAN-L2	11.36 ± 3.93
WGAN	11.80 ± 0.49
GAN-EWC	12.52 ± 6.14
GAN-ER	7.70 ± 3.94

TABLE 5.1: Symmetric KL-divergence results of the corresponding models that can be seen in the images (lower mean and SD are better).

5.3 Comparison between combinations

In this section, we tried to combine the several approaches to mitigate Mode Collapse: L2 regularization, Wasserstein GAN, Elastic Weight Consolidation, and Experience Replay. focus on the performances of how the single models behave on the; The models tested and their performances on the eight Gaussians distribution are the following: GAN with L2 and EWC 5.8, GAN with L2 and ER 5.10, Wasserstein GAN with EWC 5.9 and Wasserstein GAN with ER 5.11.

The goal is to understand if the fact that CL methods can improve the Vanilla GAN is just an exception or not. Therefore, CL methods are applied also to other GANs like GAN-L2 or WGAN: if there is an improvement, we can say with more certainty that the correlation between Mode Collapse and Catastrophic Forgetting can be true.

The chosen ER hyperparameters for the GAN with L2 and ER (5.10) are a batch size of new samples of 384, a batch size of buffer samples of 128, a buffer size of 200, 1000 as starting iteration of memorization, 0.01 of gradient clipping. Instead, for WGAN with ER (5.11) are 64 of new batch size, 32 of buffer batch size, 5000 of buffer size, 2000 as starting iteration of memorization, no clipping.

Also here, each illustration shows the 10 best trainings of that model, where the green dots are the 8 Gaussians of the real distributions while the several blue dots are samples from the generator's distribution.

Table 5.2 shows the mean and standard deviations of the symmetric KL-divergence score of the models.

Symmetric KL-divergence (Mean $\pm$ SD)			
/	None	EWC	ER
Vanilla GAN	16.43 $\pm$ 6.58	12.52 $\pm$ 6.14	7.70 $\pm$ 3.94
GAN-L2	11.36 $\pm$ 3.93	10.14 $\pm$ 2.93	9.93 $\pm$ 1.73
WGAN	11.80 $\pm$ 0.49	12.83 $\pm$ 0.95	11.13 $\pm$ 0.67

TABLE 5.2: Symmetric KL-divergence results of all the models of sections 5.2 and 5.3. The light blue cells shows the mean and SD of the models combinations (L2-EWC, L2-ER, WGAN-EWC, WGAN-ER), while the white cells are the same results of table 5.1 that are reported to have a better comparison (lower mean and SD are better).

5.4 Varying Experience Replay's buffer size

This set of experiments is focused on the Experience Replay approach and in particular on the buffer, applied to the Vanilla GAN. The goal is to understand if the increment of the buffer size leads to an increment of performances also in the case of the GAN models. Indeed, a big buffer leads to a strong Continual Learning contribution that leads to a less severe Catastrophic Forgetting problem that may lead to a less severe Mode Collapse problem.

The experiment is done by attempting several values of this hyperparameter. We tried as buffer sizes: 400, 800, 1200, 1600, 2000, 3000, 5000, 7500, 10000, 15000. The other ER hyperparameters are a batch size of new samples of 384, a batch size of buffer samples of 128, 1000 as starting iteration of memorization, 0.01 of gradient clipping. The starting memorization of ER is set to 1000 because initially, the generator's distribution alters a lot before reaching a more stable condition; therefore, it is done to not register the bad model initialization.

Table 5.3 shows the performances with mean and SD of the symmetric KL-divergence of each model with different buffer sizes.

Image 5.12 shows what the Experience Replay buffer memorizes in the 10 trainings of the GAN with ER model of image 5.7 (the hyperparameters are the same). The 8 green dots are the eight Gaussians of the real distribution while the small red dots are the samples memorized in the buffer.

Symmetric KL-divergence	
Model	Mean $\pm$ SD
400	10.64 ± 4.90
800	8.53 ± 4.04
1000	15.27 ± 5.29
1200	14.08 ± 5.74
1600	12.96 ± 4.89
2000	11.02 ± 6.44
3000	9.08 ± 3.14
5000	7.70 ± 3.93
7500	10.62 ± 6.06
10000	11.81 ± 5.04
15000	14.13 ± 8.08

TABLE 5.3: Symmetric KL-divergence results of all the GAN with ER models varying the buffer size (lower mean and SD are better).

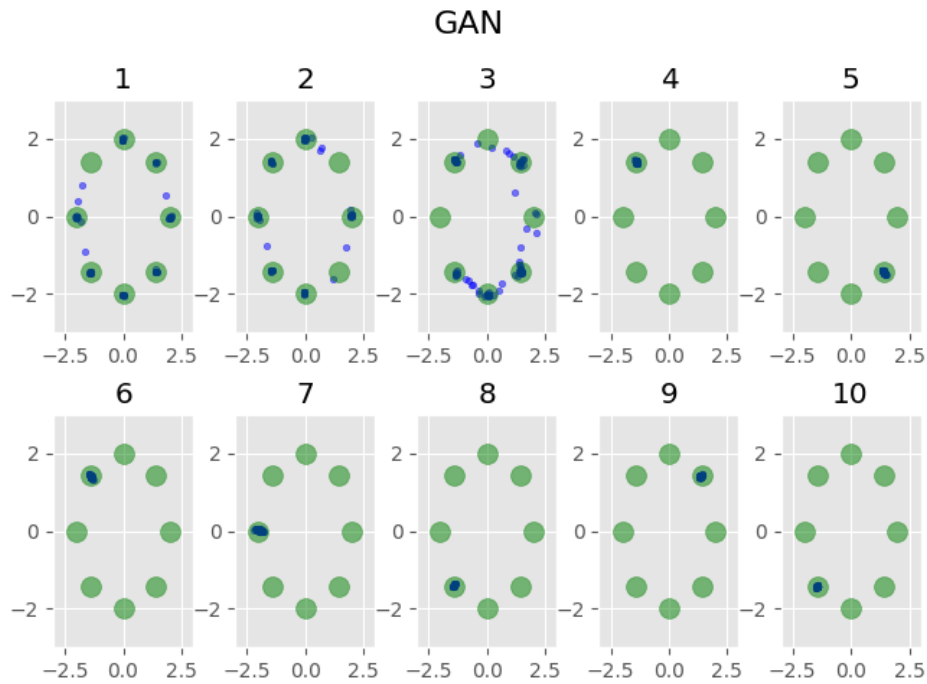


FIGURE 5.3: Illustration of 10 different trainings of the Vanilla GAN model.

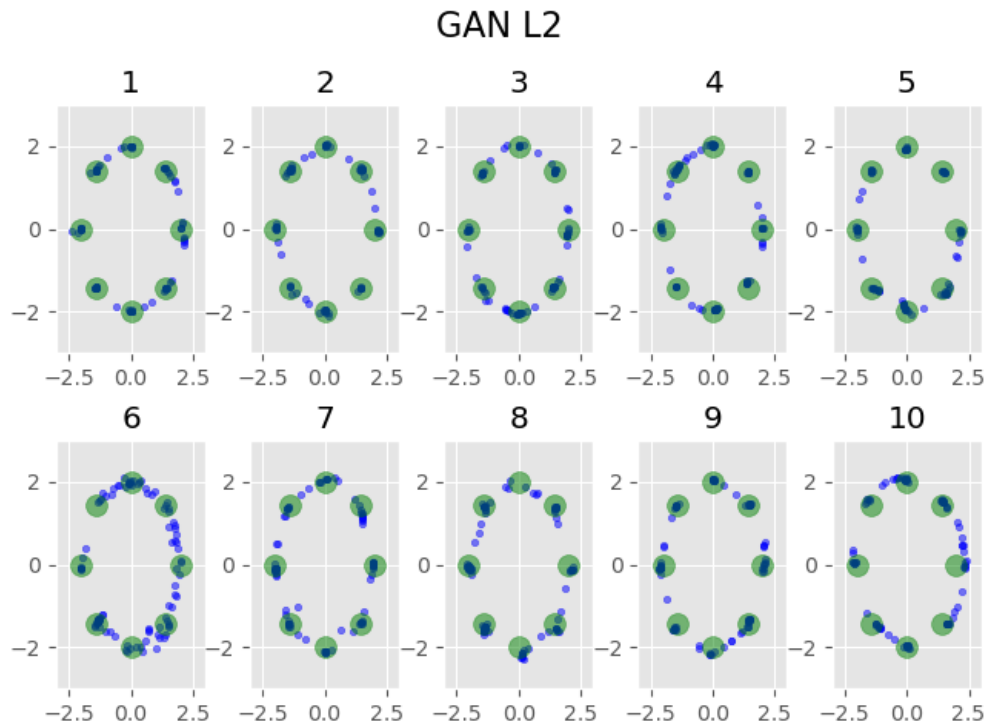


FIGURE 5.4: Illustration of 10 different trainings of the GAN with L2 regularization model.

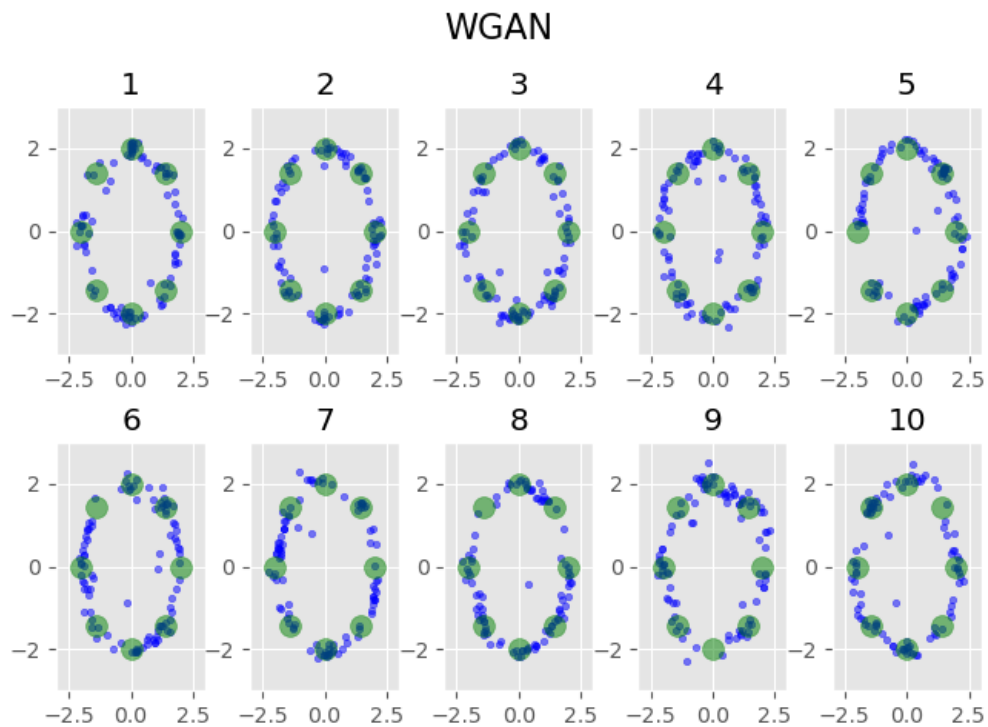


FIGURE 5.5: Illustration of 10 different trainings of the Wasserstein GAN model.

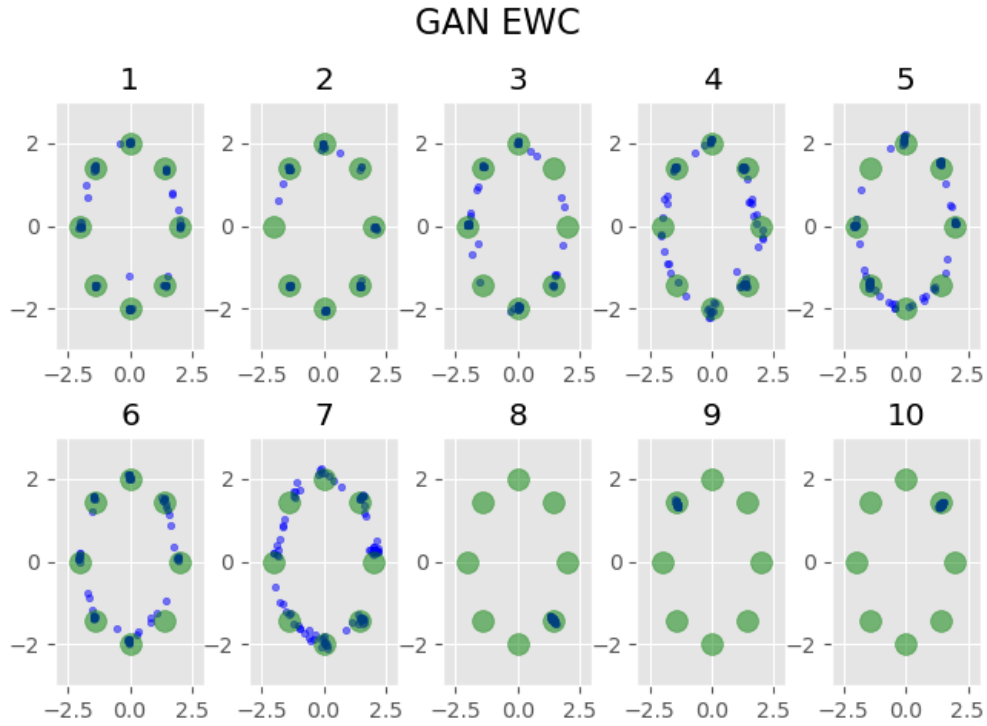


FIGURE 5.6: Illustration of 10 different trainings of the GAN with Elastic Weight Consolidation model.

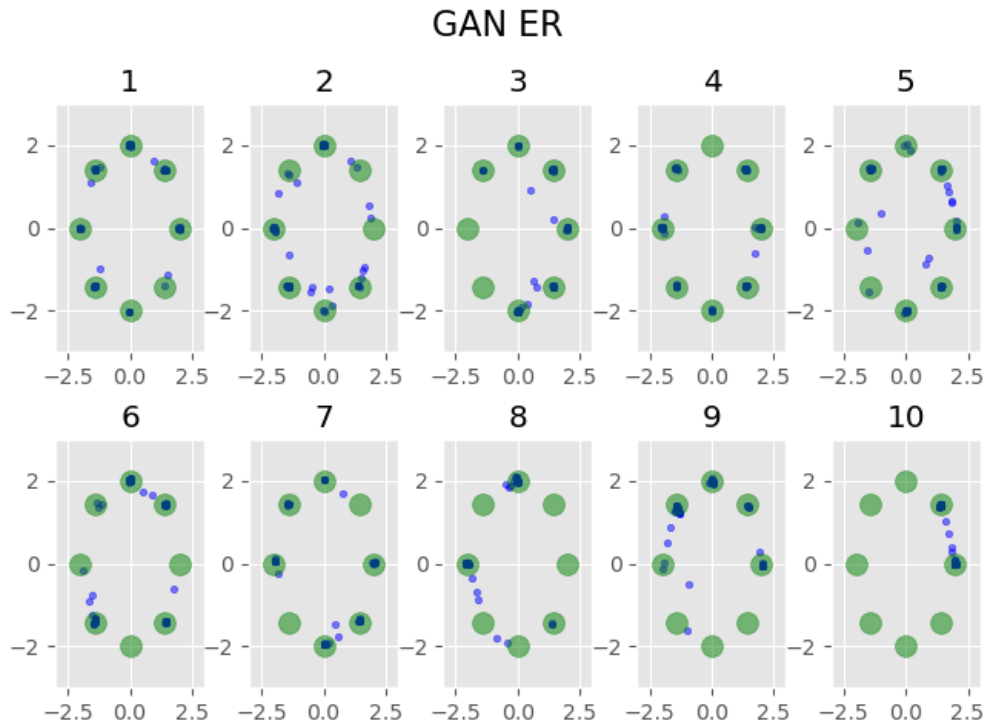


FIGURE 5.7: Illustration of 10 different trainings of the GAN with Experience Replay model.

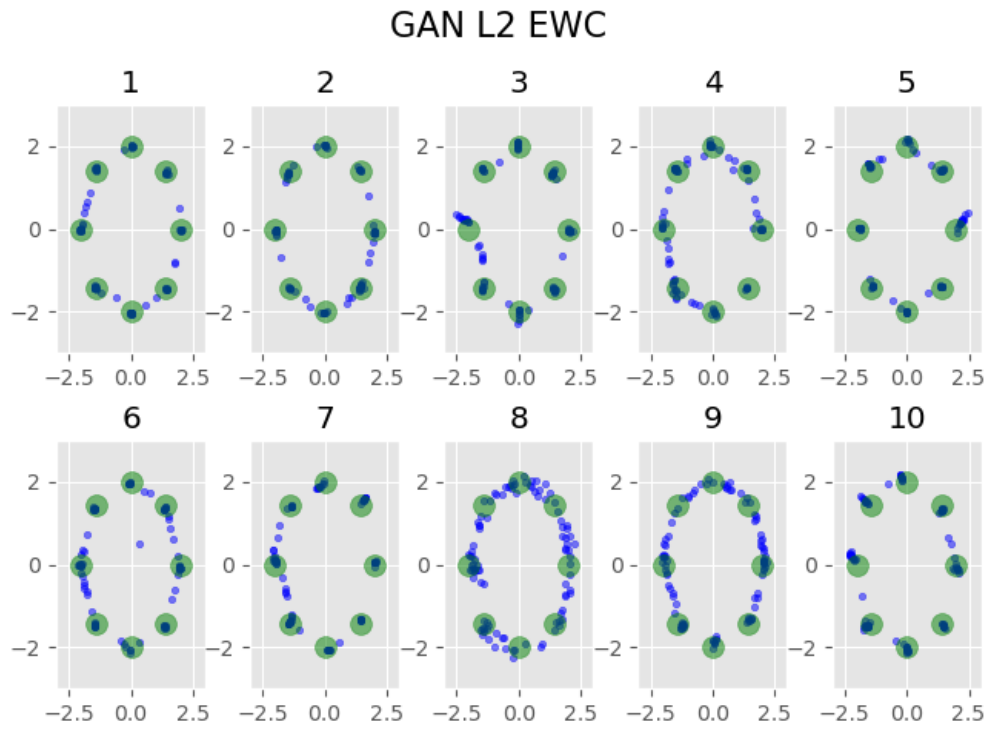


FIGURE 5.8: Illustration of 10 different trainings of the GAN model with L2 regularization and Elastic Weight Consolidation.

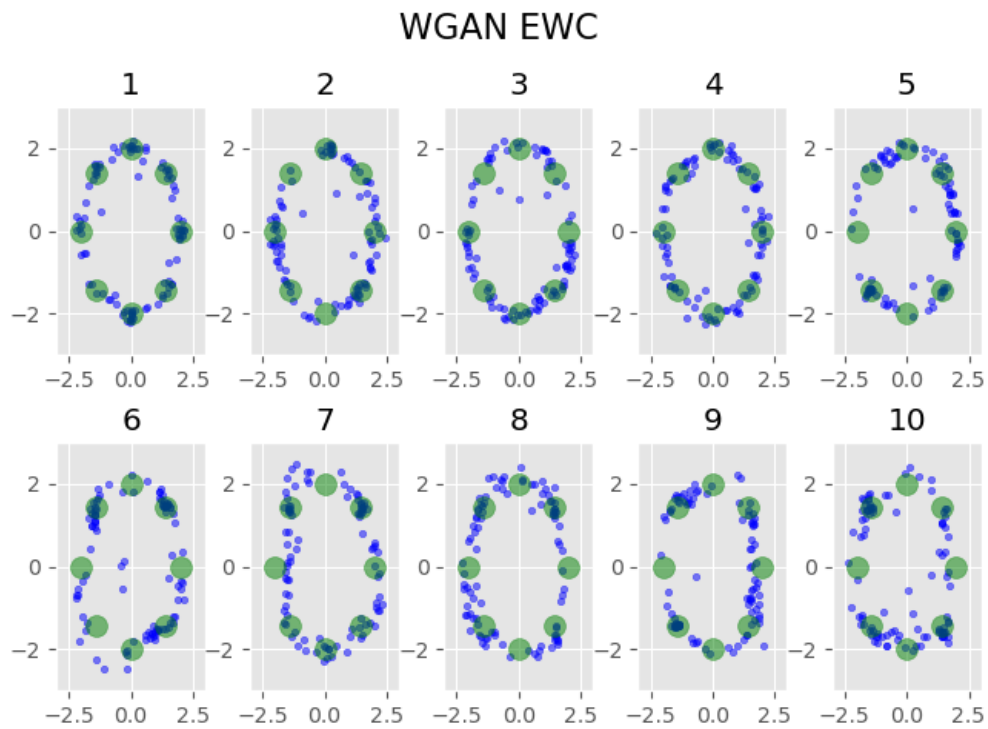


FIGURE 5.9: Illustration of 10 different trainings of the Wasserstein GAN model with Elastic Weight Consolidation.

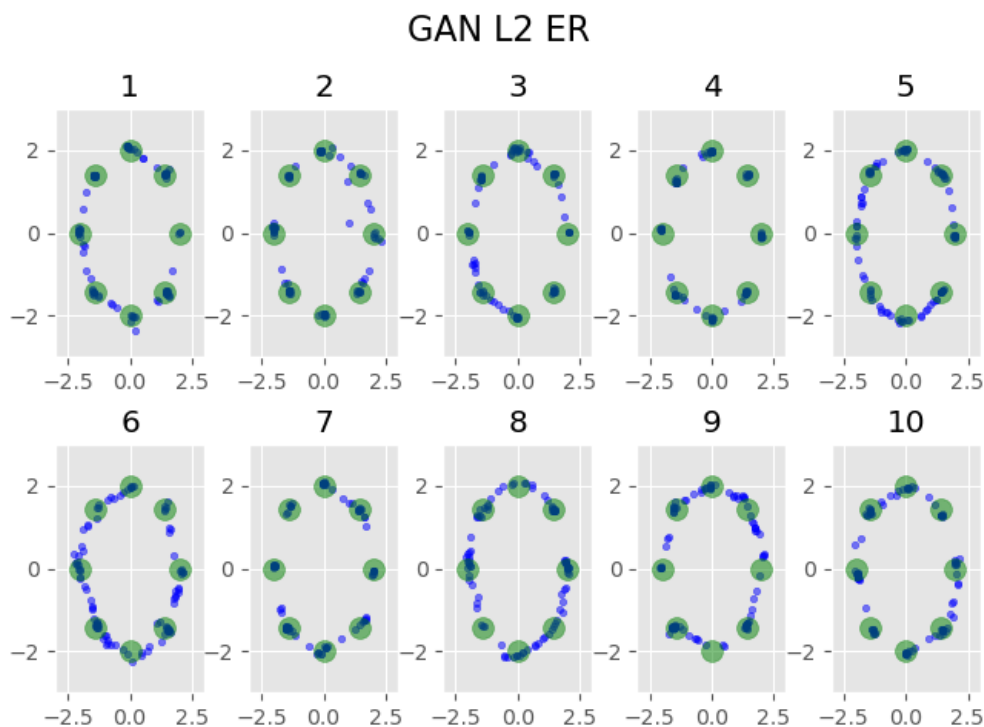


FIGURE 5.10: Illustration of 10 different trainings of the GAN model with L2 regularization and Experience Replay.

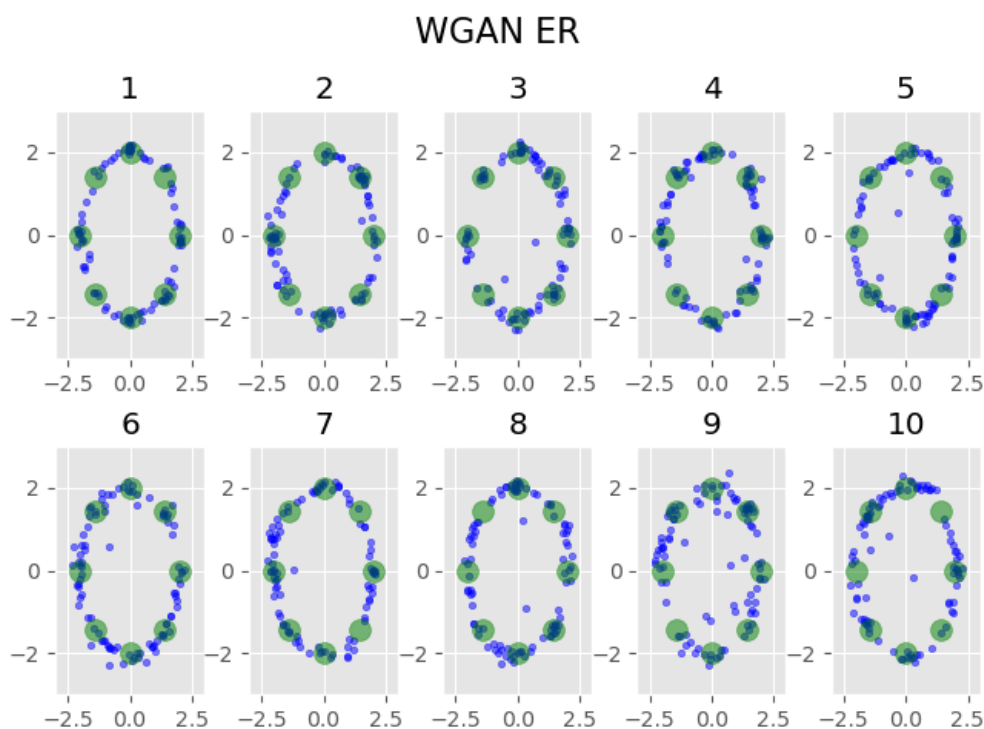


FIGURE 5.11: Illustration of 10 different trainings of the Wasserstein GAN model with Experience Replay.

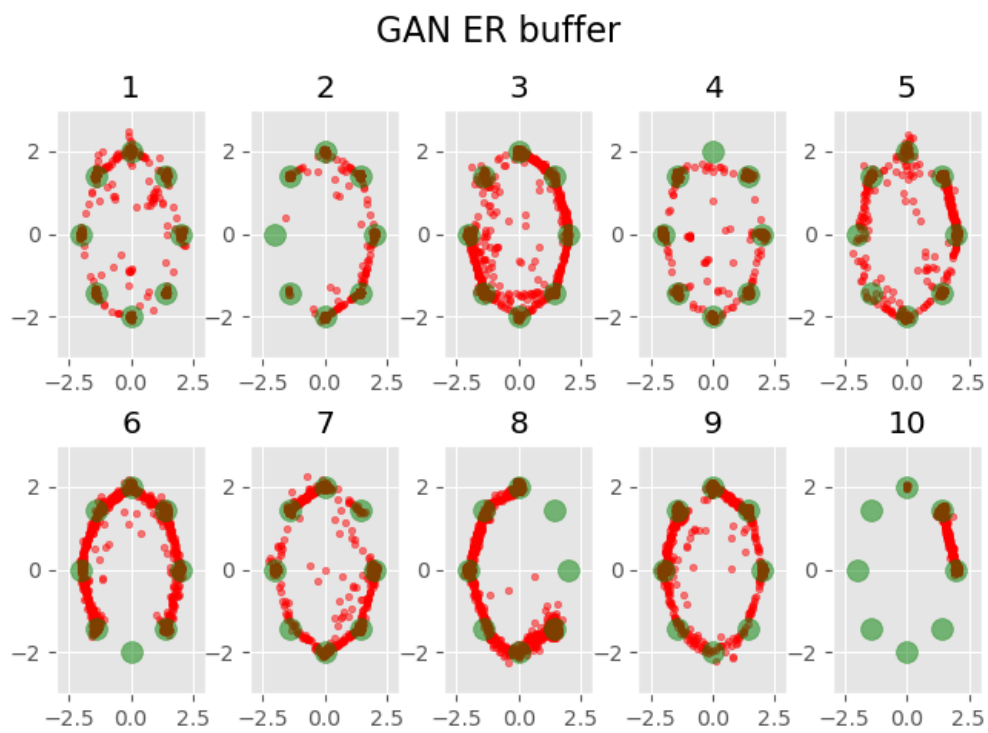


FIGURE 5.12: Illustration of 10 different trainings of the Wasserstein GAN model with Experience Replay, showing what is memorized by the buffer.

6 Discussion

In this chapter, we will discuss the tree experiments and results exhibited in the previous chapter (5).

6.1 Considerations on the different approaches

In this section, we will discuss the results of section 5.2.

We can say that the Continual Learning approaches like EWC and ER, applied to the Vanilla GAN, are able to decrease the tendency of the Generative Adversarial Networks to reach a Mode Collapse behavior. This seems to be consistent with the hypothesis of the correlation between Mode Collapse and Catastrophic Forgetting, at least in this particular contest with Vanilla GAN.

More precisely:

- **Vanilla GAN:** As we have seen in figure 5.2, the vanilla GAN model behaves as expected, since we can notice a Mode Collapse behavior where the generator moves around the eight Gaussians without spreading out on all of them. If we look at the ten trainings of the model 5.3, we can see that in the first tree the generator was able to not collapse in a single Gaussian; these three are probably fortunate trainings, where the randomness played on the generator side. All the other trainings are in a Mode Collapse. Both mean and standard deviation are quite high.
- **GAN-L2:** Thanks to the L2 regularization, the model is less prone to overfitting and therefore also to fall into the Mode Collapse problem (that, as we have seen, is correlated to the overfitting). This leads to a generator that correctly spreads around all the Gaussians, as we can see in figure 5.4. Both the mean and standard deviation are much lower with respect to the Vanilla GAN.
- **WGAN:** The Wasserstein GAN, as we can see in figure 5.5, it is able to make the training more stable and more prone to convergence, leading to the avoidance of Mode Collapse. The mean is very similar to the GAN-L2

one, but the standard deviation is much lower and it is the lowest among all the models, as we can see in table 5.1.

- **GAN-EWC:** In the GAN with EWC model, there are seven out of ten trainings where there is no Mode Collapse (figure 5.6). This is an improvement with respect to the vanilla GAN as it seems that EWC helps the model to avoid that problem. However, both mean and variance are not very low and they are worse than GAN-L2 and WGAN.
- **GAN-ER:** In the GAN with ER model, none of the ten trainings show signs of Mode Collapse, even though in the last ones the generator's distribution covers only some of the Gaussians, the last one just two (figure 5.7). If we look at the symmetric KL-divergence in table 5.1, we can see that this model has the best performance, since it has the lowest mean (7.70). The standard deviation is not very low as the generator's distribution changes considerably between trainings.

In particular, ER has higher performances than EWC, both in mean and variance; the performances of L2 and WGAN are similar looking at the mean but different in the standard deviation, probably because WGAN gives more stability than L2.

However, these methods seem to be less stable compared with L2 regularization and in particular WGAN, since the standard deviation is lower. This happens likely because these approaches are specifically designed to increase the stability and convergence of GANs.

6.2 Considerations on the combinations

In this section, we will discuss the results of section 5.3.

In general, we can say that EWC and ER applied on L2 or WGAN are not able to improve significantly their original performances. These results are valid only in this specific context: with the specific eight Gaussians distribution and the specific Vanilla GAN model. Since L2 and WGAN are able to solve Mode Collapse on their own in this simple distribution, we don't know if CL can actually improve them if we have another more complex distribution. Moreover, the evaluation method used (Sym-KL on the 10 best trainings) may be not appropriate for this assessment. Therefore, we suggest for future works to apply these combined methods on more complex distributions and to use a more suited evaluation method.

To be more specific:

- **L2-EWC:** The GAN with L2 regularization and EWC model behaves better than both GAN-L2 and GAN-EWC, as we can see the mean in table 5.2, meaning that L2 and EWC may be compatible. This compatibility supposition is enforced by the fact that L2 and EWC have very similar behavior if we look at the videos of the training progress. If we look at the trainings (figure 5.8), none of them show signs of mode collapse.
- **WGAN-EWC:** The WGAN with EWC model behaves worse than both WGAN and GAN-EWC (table 5.2), revealing that these two approaches seem to be inharmonious. The thing that improves is the standard deviations, and this is probably a contribution of WGAN since it has the lowest one. None of the ten trainings collapses (figure 5.9).
- **L2-ER:** The GAN with L2 regularization and ER model behaves better than GAN-L2 but worse than GAN-ER (table 5.2), maybe because there is an interference between the two. None of the ten trainings collapses (figure 5.10).
- **WGAN-ER:** The WGAN with ER model behaves even worse than L2-ER but still better than WGAN (table 5.2). Also here there is maybe an incompatibility between WGAN and ER. Also here none of the ten trainings collapses (figure 5.11).

Speculative analysis on the combined models behavior

If we look at all the models and their performances on table 5.2, we can understand several things.

Firstly, we can consider that the standard deviation seems to be related to the concept of stability, they are maybe inversely proportional. This can happen because the more stable a model is, the less diversity there will be between trainings and therefore also a lower standard deviation.

Focusing on the ER column of the table, we can see that, moving top-down, the model performance (mean) decreases while the stability (standard deviation) increases. This means that both L2 and WGAN can increase the stability, but at the same time they reduce the performance. This phenomenon appears to be caused by the fact that higher stability brings also a higher conservativeness of the model, limiting the ER contribution. The potentiality of ER of

reaching a better fit of the eight Gaussians is inhibited. This effect is stronger with WGAN than L2 since it has higher stability.

Focusing on the L2 regularization row of the table, moving left-right, the model behaves better (mean) and it also gains better stability (standard deviation). These results seem normal since GAN-ER is better than GAN-EWC that is better than Vanilla GAN. The L2 contribution can almost halve the standard deviation of all the three models (vanilla GAN, GAN-EWC, GAN-ER), proving its stability capabilities.

Focusing on the EWC column of the table, we can see that the comparison of these models' performances shows something odd: EWC gets better with L2 but remains almost the same with WGAN. It seems that EWC and L2 are compatible while WGAN and EWC are not very much. A possible explanation is that, by looking at the training videos, EWC and WGAN have a very different behavior (leading to a incompatibility), while EWC and L2 are very similar (leading to a compatibility). The performance (mean) of GAN-EWC and WGAN-EWC shouldn't be read in the same way since their values are determined by two different factors: in GAN-EWC the mean is raised by the fact that there are three collapsed trainings (figure 5.6) while in WGAN-EWC the mean is raised by the fact that the discriminator's model is sparsely spread in a circle around the eight Gaussians, without fitting them perfectly.

Focusing on the WGAN row of the table, we can see from the low standard deviation, the WGAN contribution gives a consistent boost of stability on all the trainings where WGAN is involved. In all the three models (WGAN, WGAN-EWC, WGAN-ER) the mean never reaches a low value since, with WGAN, the generator's distribution doesn't fit perfectly the eight Gaussians but it has a circular shape that produces samples also in between the Gaussians. Also here we can see the incompatibility between WGAN and EWC since WGAN reaches the highest values for both mean and std when paired with EWC.

6.3 Considerations on the ER buffer size

In this section, we will discuss the results of section 5.4.

As we can see in table 5.3, the performances of the GAN with ER model changes considerably varying the buffer size, in a range between 7.70 and 15.27. However, in this case, the usual trend of direct proportion between

buffer size and performance is not followed, suggesting that this trend is not true for GANs.

We can also notice that the mean values change gradually, reaching two peaks: one with a buffer size of 800 and the other with a size of 5000. These buffer size values are not universal since they are good considering the combination with the other hyperparameter values (written in section 5.4).

Looking at the samples memorized in the ER buffer in figure 5.12, we can say that ER is not able to always memorize the generator's position in all the eight Gaussians. This may happen because the buffer is constantly updated and, if the generator's distribution doesn't place on a Gaussian for a lot of training iterations, the buffer forgets these positions.

If we compare the previous figure with figure 5.7, we see that the buffer memorizes portions between Gaussians, where the current generator is not present. This may happen because ER buffer updates slower as iterations run and, after several iterations, the buffer shows positions of the generator that are far into the past.

By looking at the training videos, we may split the evolution of the generator's distribution during training in roughly three steps: in the first step, the generator moves around the eight Gaussians fitting them one at a time (Mode Collapse); in the second step, after some iterations the generator, forced by ER, spreads around the circle of Gaussians, positioning itself also in between the Gaussians; in the third step, as iterations run, the generator gets more and more accurate fitting just the eight Gaussians. We conjecture, the buffer shows the position that the generator's distribution had during the second phase, even though in the final iteration is already in the third step.

7 Conclusion

In this work we have seen that using Continual Learning methods like EWC or ER, it's possible to limit the problem of Mode Collapse, improving the performance of the GAN. This is true in particular in the Vanilla GAN case, while for GAN-L2 and WGAN this may be not true. This CL improvement is possible since, as we conjectured, there is a correlation between the problem of Mode Collapse and the problem of Catastrophic Forgetting, and therefore, solving one of the two will solve also the other.

We have also seen a comparison of these Continual Learning methods with baseline approaches that improve the convergence like L2 regularization and Wasserstein GAN. We can say that these two approaches seem to improve performances and solve Mode Collapse, but they aren't able to make the model reach the same values of performance as ER does.

In the experiments, we have also seen an attempt of combination between the baseline approaches and Continual Learning ones. Generally, there is no improvement in the baseline models but, since this result is valid only in our specific context, we suggest for future works to deepen this experiment with a different context. Overall, these models seem to show a mixture of the behaviors of the single approaches.

We also noticed that, with the specific Vanilla GAN used, the performance doesn't improve by increasing the ER buffer size, as usually happens.

The experiments done in this work can be extended by trying new and more powerful Continual Learning approaches and new combinations, with the goal of finding an even better GAN model. It is suggested to elaborate on the combined models using a more complex distribution and a more suited evaluation method. Another thing to consider is to find better hyperparameters of each approach: in this work, only some of them were tested and maybe there are better ones. It would be interesting to also use different approaches during different training iterations, for instance starting the training with WGAN and then fine-tuning with ER.

We wish that the experiments described in this paper offer a higher awareness of the Mode Collapse and Catastrophic Forgetting problems and will be

applicable for further researches.

Bibliography

- [1] @ajmooch. 2019. URL: <https://twitter.com/ajmooch/status/1182018330070306816>.
- [2] Alind, Gupta. *Modal Collapse in GANs*. 2019.
- [3] Arjovsky, Martin, Chintala, Soumith, and Bottou, Léon. “Wasserstein generative adversarial networks”. In: *International conference on machine learning*. PMLR. 2017, pp. 214–223.
- [4] Brownlee, Jason. *Master Machine Learning Algorithms: discover how they work and implement them from scratch*. Machine Learning Mastery, 2016.
- [5] Brownlee, Jason. “Crash Course On Multi-Layer Perceptron Neural Networks”. In: *Machine Learning Mastery* (2019).
- [6] Brownlee, Jason. *Generative adversarial networks with python: deep learning generative models for image synthesis and image translation*. Machine Learning Mastery, 2019.
- [7] Brownlee, Jason. “How to Identify and Diagnose GAN Failure Modes”. In: *Machine Learning Mastery* (2021).
- [8] Buzzega, Pietro et al. “Dark experience for general continual learning: a strong, simple baseline”. In: *arXiv preprint arXiv:2004.07211* (2020).
- [9] *Catastrophic Forgetting*. <https://standoutpublishing.com/g/Catastrophic-Forgetting.html>. Accessed: 2021-11-8.
- [10] Deng, L. *Deep learning: methods and applications*. *Foundations Trends Signal Process* 7 (3–4): 199–200. 2014.
- [11] Fernando, Chrisantha et al. “Pathnet: Evolution channels gradient descent in super neural networks”. In: *arXiv preprint arXiv:1701.08734* (2017).
- [12] Godoy, Daniel. “Understanding binary cross-entropy /log loss: a visual explanation”. In: *Towards datascience* (2018).
- [13] Goodfellow, Ian. “Nips 2016 tutorial: Generative adversarial networks”. In: *arXiv preprint arXiv:1701.00160* (2016).
- [14] Goodfellow, Ian, Bengio, Yoshua, and Courville, Aaron. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.

- [15] Goodfellow, Ian et al. "Generative adversarial nets". In: *Advances in neural information processing systems* 27 (2014).
- [16] Goodfellow, Ian et al. "Generative adversarial networks". In: *Communications of the ACM* 63.11 (2020), pp. 139–144.
- [17] Google. *Generative Adversarial Networks course*. 2019.
- [18] Hinton, Geoffrey, Vinyals, Oriol, and Dean, Jeff. "Distilling the knowledge in a neural network". In: *arXiv preprint arXiv:1503.02531* (2015).
- [19] Hui, Jonathan. "GAN-Why it is so hard to train Generative Adversarial Networks!" In: *Medium, Data Science* (2018).
- [20] Kingma, Diederik P and Ba, Jimmy. "Adam: A method for stochastic optimization". In: *arXiv preprint arXiv:1412.6980* (2014).
- [21] Kirkpatrick, James et al. "Overcoming catastrophic forgetting in neural networks". In: *Proceedings of the national academy of sciences* 114.13 (2017), pp. 3521–3526.
- [22] Koehrsen, Will. *Overfitting vs. Underfitting: A Conceptual Explanation*. 2018.
- [23] Li, Chengcheng, Wang, Zi, and Qi, Hairong. "Fast-converging conditional generative adversarial networks for image synthesis". In: *2018 25th IEEE international conference on image processing (ICIP)*. IEEE. 2018, pp. 2132–2136.
- [24] Li, Zhizhong and Hoiem, Derek. "Learning without forgetting". In: *IEEE transactions on pattern analysis and machine intelligence* 40.12 (2017), pp. 2935–2947.
- [25] Liang, Kevin J et al. "Generative adversarial network training is a continual learning problem". In: *arXiv preprint arXiv:1811.11083* (2018).
- [26] Lomonaco, Vincenzo. "Why Continual Learning is the Key Towards Machine Intelligence". In: *Continual AI* (2017).
- [27] Lopez-Paz, David and Ranzato, Marc'Aurelio. "Gradient episodic memory for continual learning". In: *Advances in neural information processing systems* 30 (2017), pp. 6467–6476.
- [28] Masse, Nicolas Y, Grant, Gregory D, and Freedman, David J. "Alleviating catastrophic forgetting using context-dependent gating and synaptic stabilization". In: *Proceedings of the National Academy of Sciences* 115.44 (2018), E10467–E10475.
- [29] Mayank, Tripathi. *Underfitting and Overfitting in Machine Learning*. 2020.

- [30] McCloskey, Michael and Cohen, Neal J. "Catastrophic interference in connectionist networks: The sequential learning problem". In: *Psychology of learning and motivation*. Vol. 24. Elsevier, 1989, pp. 109–165.
- [31] Metz, Luke et al. "Unrolled generative adversarial networks". In: *arXiv preprint arXiv:1611.02163* (2016).
- [32] Mitchell, Tom. "Machine learning". In: (1997).
- [33] Murphy, Kevin P. *Machine learning: a probabilistic perspective*. MIT press, 2012.
- [34] Ratcliff, Roger. "Connectionist models of recognition memory: constraints imposed by learning and forgetting functions." In: *Psychological review* 97.2 (1990), p. 285.
- [35] Rebuffi, Sylvestre-Alvise et al. "icarl: Incremental classifier and representation learning". In: *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*. 2017, pp. 2001–2010.
- [36] Riemer, Matthew et al. "Learning to learn without forgetting by maximizing transfer and minimizing interference". In: *arXiv preprint arXiv:1810.11910* (2018).
- [37] Rosenblatt, Frank. "The perceptron: a probabilistic model for information storage and organization in the brain." In: *Psychological review* 65.6 (1958), p. 386.
- [38] Schwarz, Jonathan et al. "Progress & compress: A scalable framework for continual learning". In: *International Conference on Machine Learning*. PMLR. 2018, pp. 4528–4537.
- [39] Serra, Joan et al. "Overcoming catastrophic forgetting with hard attention to the task". In: *International Conference on Machine Learning*. PMLR. 2018, pp. 4548–4557.
- [40] Shin, Hanul et al. "Continual learning with deep generative replay". In: *arXiv preprint arXiv:1705.08690* (2017).
- [41] Simone Calderara, Palazzi. "Deep Neural Networks". In: (2019).
- [42] Sternberg, Robert J. *Handbook of intelligence*. Cambridge University Press, 2000.
- [43] Van de Ven, Gido M and Tolias, Andreas S. "Three scenarios for continual learning". In: *arXiv preprint arXiv:1904.07734* (2019).

- [44] Ven, Gido M van de and Tolias, Andreas S. “Three scenarios for continual learning”. In: *arXiv preprint arXiv:1904.07734* (2019).
- [45] Wang, Qing, Kulkarni, Sanjeev R, and Verdú, Sergio. “Divergence estimation for multidimensional densities via k -nearest-neighbor distances”. In: *IEEE Transactions on Information Theory* 55.5 (2009), pp. 2392–2405.
- [46] Wu, Yue et al. “Large scale incremental learning”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2019, pp. 374–382.
- [47] Xu et al. “Infrared and Visible Image Fusion with a Generative Adversarial Network and a Residual Network”. In: *Applied Sciences* 10 (Jan. 2020), p. 554. DOI: [10.3390/app10020554](https://doi.org/10.3390/app10020554).
- [48] Yassin, Hajaj. *Introduction to Supervised, Semi-supervised, Unsupervised and Reinforcement Learning*. 2020.
- [49] Zenke, Friedemann, Poole, Ben, and Ganguli, Surya. “Continual learning through synaptic intelligence”. In: *International Conference on Machine Learning*. PMLR. 2017, pp. 3987–3995.